Language Models From First Principles

June 22, 2026

1 LLM From Scratch: Complete College-Level Walkthrough

This notebook is a single-file companion to the full `llm-from-scratch` curriculum. It follows the same 13-section journey as the notebook sequence, but it now reads more like an undergraduate machine learning course for a mathematically trained reader. The organizing rule is simple: every formula must eventually land in a tensor, a training objective, or a behavior of the toy decoder-only GPT implemented in `src/llm_from_scratch`.

The reader I have in mind is comfortable with functions, vector spaces, matrices, basic probability, and proof-style reasoning. The reader may not yet be comfortable with machine learning habits: empirical risk, stochastic optimization, logits, masking, sampling, validation loss, or the difference between a model class and a training procedure. This notebook tries to bridge those habits without hiding the algebra.

The central object is an autoregressive language model. Given a finite vocabulary of token IDs and a prefix of tokens, it returns a probability distribution for the next token. In symbols, if

$$x_{1:T} = (x_1, x_2, \dots, x_T), \qquad x_t \in \{0, 1, \dots, V-1\},$$

then the model represents conditional probabilities of the form

$$p_\theta(x_t \mid x_{<t}),$$

where θ denotes all trainable parameters: embedding tables, attention projections, MLP weights, layer norm parameters, and the final language-model head. The curriculum builds a tiny version of this map from scratch, then shows how the same ideas appear in PyTorch, Hugging Face, quantization, and modern LLM practice.

A good way to use this walkthrough:

1. Read a section once without running code.
2. Write down the mathematical object being defined.
3. Predict every tensor shape in the next code cell.
4. Run the code cell.
5. Change one small dimension or hyperparameter and explain exactly which formula or tensor contract changed.
6. Treat the exercises as self-checks, not as a separate homework set.

The notebook is intentionally CPU-friendly. It is not trying to train a useful assistant. It is trying to make the toy implementation mathematically legible.

1.1 Map To The Existing Curriculum

The original notebooks are still the best way to study one topic at a time. This file is the continuous pass. It keeps the same arc, but makes the mathematical and statistical contracts explicit.

Chapter in this notebook	Original notebook	Main mathematical question	Main code path
0. Orientation	`00_project_orientation.ipynb`	What distribution does an autoregressive model represent?	`TinyGPT.forward` returns logits for next-token prediction
1. Tensors, autograd, probability	`01_tensors_autograd_and_probability.ipynb`	How do logits become a differentiable negative log-likelihood?	`stable_softmax`, `cross_entropy_from_logits`
2. Tokenization	`02_text_tokenization_char_to_subword.ipynb`	How does text become a finite sequence over a vocabulary?	`CharTokenizer`, `train_bpe_tokenizer`
3. Embeddings and language modeling	`03_embeddings_and_language_modeling.ipynb`	How does a finite set map into a vector space, and how does one sequence become many labels?	`nn.Embedding`, `get_batch`, `TinyGPT.token_embedding`
4. Attention	`04_attention_from_raw_tensors.ipynb`	How does each position form a probability distribution over earlier positions?	`scaled_dot_product_attention`, `CausalSelfAttention`
5. Transformer block	`05_transformer_block_from_scratch.ipynb`	How are residual maps, normalization, attention, and MLPs composed?	`GPTBlock`, `FeedForward`, `TinyGPT`
6. Training	`06_training_loop_loss_and_optimization.ipynb`	How does empirical risk minimization change parameters?	`train_steps`, `overfit_tiny_batch`
7. Generation and evaluation	`07_generation_sampling_and_evaluation.ipynb`	How does a trained conditional distribution produce a sequence?	`generate`, `sample_next_token`, `perplexity`
8. Toy fine-tuning	`08_finetuning_toy_instruction_task.ipynb`	How does masking turn some positions into supervised targets and others into context?	`toy_instruction_examples`; masked risk illustrated directly
9. Library translation	`09_hugging_face_translation_layer.ipynb`	Which from-scratch objects become library abstractions?	`datasets`, `tokenizers`, `GPT2LMHeadModel`
10. Quantization	`10_quantization_deep_dive.ipynb`	How does a real tensor map to a finite integer grid?	`quantize_tensor`, `dequantize_tensor`, `estimate_kv_cache_bytes`
11. Modern orientation	`11_modern_llm_orientation.ipynb`	How do objectives, data distributions, retrieval, evaluation, and serving wrap the core model?	Orientation map in this notebook

Chapter in this notebook	Original notebook	Main mathematical question	Main code path
12. Beyond transformers	`12_beyond_transformers_and_world_models.ipynb`	Which mathematical bottleneck is a new architecture trying to relax?	Attention cost comparison in this notebook
13. Study path	this notebook	How should a mathematical reader keep studying without losing the implementation thread?	Exercises and detailed notebooks

1.1.1 Source note

This revision cross-checks the curriculum against Tong Xiao and Jingbo Zhu, *Foundations of Large Language Models* (arXiv:2501.09223v2, CC BY 4.0). The additions are paraphrased and folded into this repo's toy implementation rather than copied as textbook excerpts. The most useful imported emphases are: causal language modeling as right-context masking, the distinction between prefill and decoding during inference, KV-cache sharing through MQA/GQA, RAG/tool-use as problem decomposition, and DPO as a preference objective that avoids an explicit reward model.

The repeating pattern is:

1. State the mathematical object.
2. Attach dimensions and tensor shapes.
3. Explain its statistical or optimization role.
4. Derive the formula at an undergraduate level.
5. Point to the exact toy code path.
6. Interpret the abstraction.
7. Warn about common ML failure modes.

When a topic does not satisfy that pattern, it is probably still too vague.

```
from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().resolve().parent if Path.cwd().name == "notebooks" else Path.cwd().resolve()
SRC_DIR = PROJECT_ROOT / "src"
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))

PROJECT_ROOT
```

1.2 0. What A Language Model Computes

1.2.1 Mathematical object

A decoder-only language model is a parameterized family of conditional distributions over a finite vocabulary. Let the vocabulary be

$$\mathcal{V} = \{0, 1, \dots, V-1\}.$$

A token sequence of length T is an element of $\mathcal{V}^T$. The autoregressive modeling assumption uses the chain rule of probability:

$$P_\theta(X_{1:T} = x_{1:T}) = \prod_{t=1}^{T} P_\theta(X_t = x_t \mid X_{<t} = x_{<t}).$$

This is not an approximation by itself. It is the exact chain rule. The modeling choice is that we build one neural network that estimates each conditional factor from the prefix. In practice we often include a special beginning-of-sequence token, so the first factor also has a context.

Taking logs gives

$$\log P_\theta(x_{1:T}) = \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}).$$

Maximum likelihood training chooses parameters that make observed sequences likely. Equivalently, it minimizes negative log-likelihood:

$$-\log P_\theta(x_{1:T}) = \sum_{t=1}^{T} -\log P_\theta(x_t \mid x_{<t}).$$

This sum is the first reason language modeling becomes a per-token supervised learning problem. One text sequence yields many prediction terms.

1.2.2 Causal language modeling among pretraining tasks

The textbook's useful framing is that decoder-only next-token training is one member of a broader family of self-supervised pretraining tasks. Causal language modeling can be understood as a mask-predict task where every position is prevented from using its right context. For token x_t, the visible context is $x_{<t}$ and the hidden part is the future.

Masked language modeling, used by encoder-style systems, chooses a subset of token positions $A(x)$, replaces those positions with a mask symbol, and predicts only the masked tokens from the corrupted sequence. Its objective has the schematic form

$$\sum_{i \in A(x)} \log p_\theta(x_i \mid \bar{x}),$$

where $\bar{x}$ is the corrupted input. The important contrast is context direction. A causal decoder predicts from left context only. A masked encoder can use both left and right unmasked context. Permuted language modeling changes the prediction order while preserving original token positions, usually by changing attention masks.

This notebook keeps the decoder-only case because it is the path implemented by `TinyGPT`: a lower-triangular mask, next-token targets, and logits for every position.

1.2.3 Dimensions and tensor shapes

In the toy model, a batch of input token IDs has shape `(B, T)`:

$$\texttt{idx}[b,t] \in \{0,1,\dots,V-1\}.$$

The model returns logits with shape `(B, T, V)`. The vector `logits[b, t, :]` contains one real score for each vocabulary item as the target for position `t`. After softmax, it becomes a probability vector in the $(V-1)$-simplex:

$$\Delta^{V-1} = \left\{ p \in \mathbb{R}^V : p_i \geq 0,\ \sum_{i=0}^{V-1} p_i = 1 \right\}.$$

The code path is `TinyGPT.forward` in `src/llm_from_scratch/model.py`. It computes token and position embeddings, applies transformer blocks, normalizes the residual stream, and applies `lm_head` to obtain logits:

```
logits = self.lm_head(x)  # shape (B, T, V)
```

If targets are supplied, the same method calls PyTorch cross-entropy on flattened token positions:

```
loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
```

Flattening from `(B, T, V)` to `(B*T, V)` means every batch-position pair becomes one multiclass classification example.

1.2.4 Statistical role

The statistical problem is not to memorize a deterministic next token function. Natural language is not a function from prefixes to unique continuations. The same prefix can have many plausible next tokens. The model estimates a conditional distribution. Training asks whether the observed next token received high probability under that distribution.

For a dataset of token sequences, the empirical risk is the average negative log-likelihood over observed positions:

$$\widehat{R}(\theta) = \frac{1}{N} \sum_{n=1}^{N} \frac{1}{T_n - 1} \sum_{t=1}^{T_n - 1} -\log p_\theta(x_{t+1}^{(n)} \mid x_{1:t}^{(n)}).$$

The toy batching code samples fixed windows, so the implementation-level version is closer to

$$\widehat{R}_{\text{batch}}(\theta) = \frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} -\log p_\theta(y_{b,t} \mid x_{b,1:t}).$$

That is exactly the scalar loss used for backpropagation.

1.2.5 Mathematician's lens

The model is a map from a finite prefix space into a probability simplex. The transformer is one parameterization of that map. Attention, embeddings, MLPs, and layer norms are not separate statistical models; they are coordinate-level machinery used to compute the conditional probability vector.

1.2.6 ML practitioner's warning

A low training loss only says the model assigned high probability to observed tokens under the training distribution. It does not prove factuality, reasoning, instruction-following, calibration, robustness, or safe behavior. Those require additional data, objectives, evaluations, and constraints.

1.2.7 Self-checks

1. If logits have shape `(8, 16, 100)`, how many multiclass classification examples are used by the cross-entropy call after flattening?
2. Why is the sequence log-likelihood a sum but the sequence probability a product?
3. Where in `TinyGPT.forward` does the model become a distribution over vocabulary items rather than a hidden representation?

1.2.8 Classical Baseline: N-Grams Before Neural LMs

Before transformers, the cleanest language-model baseline was the n-gram model. It uses the same chain-rule target as the neural model, but it replaces the full prefix with a short fixed history and estimates probabilities from counts.

The exact chain rule is

$$P(x_{1:T}) = \prod_{t=1}^{T} P(x_t \mid x_{<t}).$$

An n-gram model approximates each factor by conditioning only on the previous $n-1$ tokens:

$$P(x_t \mid x_{<t}) \approx P(x_t \mid x_{t-n+1:t-1}).$$

The maximum-likelihood count estimate has the form

$$\hat{P}(w_t \mid h) = \frac{\text{count}(h, w_t)}{\text{count}(h)},$$

where h is the history, such as the previous one or two tokens. This simple ratio exposes three issues that still matter for LLMs:

1. **Data sparsity.** Many valid histories never appear in a finite training corpus.
2. **Generalization.** A model must assign probability to held-out text, not just memorized training strings.
3. **Evaluation protocol.** Train, development, and test splits must stay separate, or probability-based metrics become misleading.

Neural LMs keep the same autoregressive question but replace sparse count tables with learned embeddings, attention, nonlinear layers, and gradient descent. The transformer is therefore not a new probability problem. It is a more flexible estimator of the same conditional factors.

Source note: this bridge follows the CS124 n-gram material and SLP3 Chapter 3, especially the chain-rule, train/test, smoothing, and perplexity discussion in `../data/cs124/slp3_chapters/3.pdf` and `../data/cs124/lectures/lm_jan25.pdf`.

1.2.9 Smoothing: The Classical Fix For Sparse Counts

The maximum-likelihood n-gram estimate is brutally literal:

$$\hat{P}(w \mid h) = \frac{\text{count}(h, w)}{\text{count}(h)}.$$

If a valid continuation never appears after history h in the training corpus, this estimate assigns it probability zero. In a sequence model, one zero factor makes the whole sequence probability zero and the log-likelihood unusable. This is not just an old n-gram problem. It is the finite-data problem in its simplest form.

Classical smoothing changes the estimate so the model reserves some probability mass for rare or unseen events. Three common moves are:

Method	Core idea	Undergrad intuition
Additive smoothing	add a small pseudo-count before normalizing	pretend every event had at least tiny evidence
Interpolation	mix higher-order and lower-order n-gram estimates	trust long histories when available, fall back when sparse
Backoff	use a shorter history when the longer history is unreliable	if `in San` is too specific, try just `San` or the unigram

Kneser-Ney smoothing adds a deeper idea: a word's usefulness is not only how often it appears, but how many different contexts it can complete. A token like `Francisco` may be frequent after `San` but not a generally likely continuation everywhere.

The neural version of this story is parameter sharing. Embeddings, attention heads, and MLPs let evidence from one context affect predictions in related contexts. They do not remove the need for held-out evaluation; they are a more flexible way to generalize beyond exact count tables.

Source note: this section comes from the CS124 n-gram and smoothing material in `../data/cs124/slp3_chapters/3.pdf`, the Kneser-Ney appendix `../data/cs124/slp3_chapters/C.pdf`, and `../data/cs124/lectures/lm_jan25.pdf`.

1.2.10 The Three Levels We Will Keep Separating

A recurring discipline in this notebook is to separate three levels of description.

Level	Example statement	What can go wrong if ignored
Mathematical object	Softmax maps $z \in \mathbb{R}^V$ to $p \in \Delta^{V-1}$.	You may forget which axis is normalized.
Tensor implementation	`logits` has shape `(B, T, V)` and softmax acts on `dim=-1`.	You may normalize over positions instead of vocabulary.
Software abstraction	`TinyGPT.forward(idx, targets)` returns `(logits, loss)`.	You may misunderstand what the API has already shifted or flattened.

A mathematically correct sentence can still be insufficient for programming. For example, saying "attention computes softmax$(QK^T)V$" omits the batch dimension, head dimension, mask broadcasting, and

the axis of softmax. In this repo, the single-head teaching function `scaled_dot_product_attention` works with tensors shaped `(B, T, D)`, while the module `CausalSelfAttention` reshapes a `(B, T, C)` residual stream into `(B, H, T, D)`.

We will use the following notation throughout:

Symbol	Meaning	Implementation name
B	batch size	first dimension of `idx`
T	context length, block size, or sequence length	second dimension of `idx`
V	vocabulary size	`config.vocab_size`
C	embedding width, channel dimension	`config.n_embd`
H	number of attention heads	`config.n_head`
D	per-head dimension	`C // H`
θ	all trainable parameters	`model.parameters()`

Two shape habits prevent many mistakes:

1. Name the semantic role of every axis before coding.
2. Check that any reduction, softmax, mean, or gather is applied along the intended axis.

For example, cross-entropy over vocabulary should reduce the `V` axis. Averaging the loss should reduce over token positions and batch examples. Causal masking should modify the source-position axis for each receiving position.

> **Mathematician's lens.** Many neural network operations are familiar maps between finite-dimensional vector spaces, but the product structure of the space matters. A tensor in $\mathbb{R}^{B\times T\times C}$ is not just a vector in $\mathbb{R}^{BTC}$ when the operation is supposed to preserve batch examples, respect causal order, or normalize over vocabulary.
>
> **ML practitioner's warning.** Shape-compatible code can still be semantically wrong. Broadcasting may make an incorrect mask run without error. A loss may decrease while the model is learning from leaked future tokens or shifted labels.

```python
import math
import torch

from llm_from_scratch.configs import ModelConfig, TrainConfig, set_seed, get_device

set_seed(123)
device = torch.device("cpu")  # CPU keeps this notebook predictable and lightweight.
print("Project device preference would choose:", get_device())
print("This walkthrough executes small demonstrations on:", device)
```

1.3 1. Tensors, Autograd, And Probability

A tensor is a rectangular array together with a shape. A differentiable program is a composition of tensor operations for which automatic differentiation can compute derivatives. A probabilistic classifier is a differentiable program whose output can be interpreted as a probability distribution. A language model is a probabilistic classifier repeated over token positions with shared parameters and causal context.

The common LLM shapes are:

Symbol	Meaning	Typical tensor shape
`B`	batch size	number of independent windows processed together
`T`	sequence length or context length	number of token positions per window
`C`	channel or embedding dimension	vector width inside the model
`V`	vocabulary size	number of possible token IDs
`H`	number of attention heads	parallel attention subspaces
`D`	per-head dimension	usually `C / H`

If `idx` has shape `(B, T)`, it contains integer token IDs. If `x = embedding(idx)`, then `x` has shape `(B, T, C)`. If the model returns logits for next-token prediction, logits have shape `(B, T, V)`.

1.3.1 Empirical risk as the scalar objective

Autograd needs a scalar objective. In supervised learning, the scalar is usually an empirical average of per-example losses. For language modeling with a batch of windows, the loss has the form

$$L(\theta) = \frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} \ell\left(z_\theta(b,t), y_{b,t}\right),$$

where $z_\theta(b,t) \in \mathbb{R}^V$ is the logit vector at one token position and $y_{b,t}$ is the target token ID. For cross-entropy,

$$\ell(z,y) = -\log \operatorname{softmax}(z)_y.$$

The toy function `cross_entropy_from_logits` implements this directly for logits with a final vocabulary axis. The model class `TinyGPT` uses PyTorch's optimized `F.cross_entropy`, but the mathematical object is the same.

1.3.2 Autograd's role

Let the model be a composition of differentiable maps:

$$\theta \mapsto z_\theta \mapsto L(\theta).$$

Backpropagation computes $\nabla_\theta L$, the gradient of the scalar loss with respect to each trainable tensor. The optimizer then chooses an update rule. Plain gradient descent would be

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta L(\theta_k),$$

where $\eta > 0$ is the learning rate. The repo uses AdamW for training helpers, but the conceptual direction is still: change parameters to reduce the empirical negative log-likelihood.

1.3.3 Code path

The code cell below uses a vector `w` and a scalar `loss = mean(w_i^2)` because it isolates the mechanism. Later, `TinyGPT.forward` creates a scalar loss from `(B, T, V)` logits and `(B, T)` targets. In both cases, `.backward()` applies the chain rule to the tensor operations that produced the scalar.

> **Mathematician's lens.** Autograd does not eliminate derivatives. It represents a large composed function and applies the chain rule mechanically. You should still know what scalar function is being differentiated.
>
> **ML practitioner's warning.** If the scalar loss averages over the wrong elements, uses misaligned labels, or includes tokens that should be masked out, autograd will faithfully optimize the wrong objective.

1.3.4 Self-checks

1. Why must the training objective be scalar before `.backward()`?
2. If `logits` has shape `(B, T, V)`, why does cross-entropy treat the final axis differently from the first two?
3. In the empirical risk formula, what changes when prompt tokens are masked during supervised fine-tuning?

1.3.5 Broadcasting And Scalar Objectives

Broadcasting is a rule for applying operations to tensors whose shapes are compatible but not identical. For example, subtracting a tensor of shape `(B, T, 1)` from a tensor of shape `(B, T, V)` subtracts one value from every vocabulary logit at the same batch-position pair. This is exactly the kind of operation used in numerically stable softmax, where the maximum logit is subtracted from every logit in the same row.

The key point is that broadcasting is syntactic convenience, not mathematical forgiveness. You still need to know which mathematical object is being copied along which axis. In attention, a causal mask of shape `(T, T)` can broadcast across batch and head dimensions to mask scores of shape `(B, H, T, T)`. That is correct only because every batch example and head obeys the same causal ordering.

The scalar objective in the code cell is

$$L(w) = \frac{1}{3}\sum_{i=1}^{3} w_i^2.$$

Its derivative is

$$\frac{\partial L}{\partial w_i} = \frac{2w_i}{3}.$$

The cell verifies that PyTorch stores this derivative in `w.grad`. This small check mirrors what happens in training: the loss is more complex, the parameters are matrices rather than a length-3 vector, but the chain rule contract is the same.

1.3.6 Connection to the toy LLM

In `TinyGPT`, parameters appear in embedding tables, linear projections, layer norms, and the final head. The loss is the average cross-entropy over token positions. Calling `loss.backward()` populates gradients for all tensors that contributed to that loss. The training helper `train_steps` then calls:

```
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

This three-line pattern is the optimization heartbeat of the toy model.

> **Mathematician's lens.** The training loop repeatedly evaluates a scalar-valued function on a high-dimensional parameter space and follows local derivative information.
>
> **ML practitioner's warning.** A nonzero gradient does not guarantee useful learning. Gradients can point in noisy directions, explode, vanish, or optimize a flawed data pipeline.

```python
w = torch.tensor([1.0, -2.0, 3.0], requires_grad=True)
loss = (w ** 2).mean()
loss.backward()

# For mean(w_i^2), the derivative is 2*w_i / 3.
expected_grad = 2 * w.detach() / w.numel()
assert torch.allclose(w.grad, expected_grad)
loss.item(), w.grad
```

1.3.7 Softmax: Scores To Probabilities

1.3.8 Mathematical object

A logit vector is a vector of unconstrained real scores:

$$z = (z_1, \dots, z_V) \in \mathbb{R}^V.$$

A probability vector over the vocabulary must live in the simplex:

$$\Delta^{V-1} = \{p \in \mathbb{R}^V : p_i \geq 0,\ \sum_i p_i = 1\}.$$

Softmax is the map

$$\operatorname{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{V} \exp(z_j)}.$$

The numerator makes every component positive. The denominator normalizes the components to sum to 1.

1.3.9 Shift invariance

For any constant c,

$$\operatorname{softmax}(z + c\mathbf{1})_i = \frac{e^{z_i + c}}{\sum_j e^{z_j + c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \operatorname{softmax}(z)_i.$$

This identity is not a trick; it is an exact invariance. The implementation uses it for numerical stability by choosing $c = -\max_j z_j$:

$$\text{softmax}(z)_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}, \qquad m = \max_j z_j.$$

Now the largest exponent is $e^0 = 1$, so overflow is less likely.

1.3.10 Log-sum-exp

Cross-entropy often needs $\log \sum_j e^{z_j}$. Direct exponentiation can overflow, so the stable identity is

$$\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}, \qquad m = \max_j z_j.$$

The toy `cross_entropy_from_logits` uses PyTorch's `torch.logsumexp`, which implements this stable pattern.

1.3.11 Jacobian

Let $p_i = \text{softmax}(z)_i$. The derivative of p_i with respect to z_k is

$$\frac{\partial p_i}{\partial z_k} = p_i(\mathbf{1}[i = k] - p_k).$$

Equivalently, the Jacobian is

$$J = \text{diag}(p) - pp^T.$$

This matrix has an important interpretation. Increasing one logit increases its own probability but must decrease some probability mass elsewhere because the probabilities sum to 1. The rows and columns reflect the coupling created by normalization.

1.3.12 Dimensions and tensor shapes

For language modeling, logits usually have shape `(B, T, V)`. Softmax should be applied along the vocabulary axis `dim=-1`, producing probabilities of the same shape. For every fixed `(b,t)`, the vector `probs[b,t,:]` sums to 1.

The notebook code calls `stable_softmax(logits, dim=-1)` from `src/llm_from_scratch/functional.py`. That function subtracts the row maximum, exponentiates, and divides by the row sum.

> **Mathematician's lens.** Softmax is a smooth map from an unconstrained vector space to the interior of a simplex. It turns additive logit differences into multiplicative odds ratios.
>
> **ML practitioner's warning.** Softmax probabilities are not automatically calibrated probabilities. A model can assign confident probabilities for bad reasons, especially off distribution or after overfitting.

1.3.13 Self-checks

1. Prove that adding the same constant to every logit does not change softmax.
2. If all logits are equal, what probability does each vocabulary item receive?
3. Why would applying softmax over `dim=1` be wrong for a `(B, T, V)` language-model logit tensor?

```
from llm_from_scratch.functional import stable_softmax, cross_entropy_from_logits

logits = torch.tensor([[1.0, 2.0, 3.0]])
probs = stable_softmax(logits, dim=-1)

assert probs.shape == logits.shape
assert torch.allclose(probs.sum(dim=-1), torch.ones(1))
assert torch.argmax(probs, dim=-1).item() == 2
probs
```

1.3.14 Cross-Entropy As Negative Log-Likelihood

1.3.15 Mathematical object

For one token-position example, the target is a class index $y \in \{1, \dots, V\}$. The model outputs logits $z \in \mathbb{R}^V$ and probabilities $p = \text{softmax}(z)$. The likelihood assigned to the observed class is p_y. The negative log-likelihood loss is

$$L(z, y) = -\log p_y.$$

Substituting softmax gives

$$L(z, y) = -\log\left(\frac{e^{z_y}}{\sum_j e^{z_j}}\right) = -z_y + \log \sum_j e^{z_j}.$$

This form is what stable implementations use: gather the correct logit and subtract it from a log-sum-exp term.

1.3.16 Gradient derivation

For $L(z, y) = -z_y + \log \sum_j e^{z_j}$,

$$\frac{\partial L}{\partial z_i} = -\mathbf{1}[i = y] + \frac{e^{z_i}}{\sum_j e^{z_j}} = p_i - \mathbf{1}[i = y].$$

So the gradient is dense. Even though the label names one correct token, every logit receives a derivative. The correct class receives $p_y - 1$, which is negative unless $p_y = 1$, so gradient descent increases that logit. Incorrect classes receive p_i, so gradient descent decreases them in proportion to their assigned probability.

1.3.17 KL and information view

Let q be a target distribution and p be the model distribution. Cross-entropy is

$$H(q, p) = -\sum_i q_i \log p_i.$$

It decomposes as

$$H(q, p) = H(q) + \mathrm{KL}(q \| p),$$

because

$$\mathrm{KL}(q \| p) = \sum_i q_i \log \frac{q_i}{p_i} = \sum_i q_i \log q_i - \sum_i q_i \log p_i = -H(q) + H(q, p).$$

For a one-hot target, $q_y = 1$ and $q_i = 0$ for $i \neq y$, so

$$H(q, p) = -\log p_y.$$

The entropy $H(q)$ of a one-hot target is 0, so minimizing cross-entropy is the same as minimizing $\mathrm{KL}(q \| p)$ for that token. For empirical language modeling, the dataset gives observed samples from an unknown data distribution. The average negative log-likelihood estimates how well the model distribution matches those observed conditional token frequencies.

1.3.18 Dimensions and code path

`cross_entropy_from_logits(logits, targets)` accepts logits whose last dimension is vocabulary. If logits have shape `(B, T, V)` and targets have shape `(B, T)`, it flattens them to `(B*T, V)` and `(B*T)`. The final scalar is the mean loss over token positions.

`TinyGPT.forward` uses `F.cross_entropy` with the same flattening idea. This is the implementation version of

$$\frac{1}{BT} \sum_{b,t} -\log p_\theta(y_{b,t} \mid x_{b,\leq t}).$$

> **Mathematician's lens.** Cross-entropy measures the expected code length under the model distribution when the data are drawn from the target distribution. For one-hot labels, it reduces to the negative log probability assigned to the observed class.
>
> **ML practitioner's warning.** Cross-entropy rewards probability assigned to observed tokens. If the data contain leakage, duplication, bad formatting, or harmful targets, the loss will optimize those patterns too.

1.3.19 Self-checks

1. Derive $\partial L / \partial z_i$ from $L = -z_y + \log \sum_j e^{z_j}$.
2. Why does the gradient sum to zero across logits?
3. In a language-model batch, what does one row of the flattened `(B*T, V)` tensor correspond to?

1.3.20 Logistic Regression As The One-Logit Warmup

Binary logistic regression is the smallest version of the same idea. It computes one real-valued score

$$z = w \cdot x + b,$$

then maps it to a probability with the sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

A vocabulary softmax generalizes this from one logit to a vector of V logits. The binary model asks, "how much evidence is there for class 1?" The language model asks, "how much evidence is there for each possible next token?"

This bridge is useful because it separates probability from action. A probability distribution is not yet a decision. In binary classification, one still chooses a threshold. In generation, one still chooses a decoding policy. In both cases, the model's probabilities become behavior only after an extra rule is applied.

Source note: this follows the CS124 logistic-regression lab's logit/sigmoid framing and its warning that probability and decision thresholds are different objects: `../data/cs124/labs/Lab2_LogisticRegression.md`.

```
logits = torch.tensor([[1.0, 2.0, 3.0]], requires_grad=True)
target = torch.tensor([2])
loss = cross_entropy_from_logits(logits, target)
loss.backward()

with torch.no_grad():
    p = stable_softmax(logits, dim=-1)
    expected_grad = p.clone()
    expected_grad[0, target.item()] -= 1.0

assert torch.allclose(logits.grad, expected_grad, atol=1e-6)
loss.item(), logits.grad
```

1.4 2. Tokenization: Text To Integer IDs

1.4.1 Mathematical object

Neural networks in this curriculum do not operate on strings directly. They operate on finite sequences of integers. A tokenizer is a pair of maps between text and token IDs:

$$\text{encode} : \text{strings} \to \mathcal{V}^T,$$

and, when possible,

$$\text{decode} : \mathcal{V}^T \to \text{strings}.$$

For a character tokenizer, the vocabulary is a finite set of characters found in the corpus. Each character is assigned an integer ID. For a subword tokenizer such as BPE, the vocabulary contains learned string fragments. The output is still a finite integer sequence.

1.4.2 Dimensions and tensor shapes

After tokenization, a text sample becomes a 1D sequence of token IDs:

`tokens.shape == (N,)`

Batching later slices this into windows:

```
x.shape == (B, T)
y.shape == (B, T)
```

The vocabulary size `V` determines the number of rows in the token embedding table and the number of logits emitted at each position.

1.4.3 Statistical role

Tokenization defines the sample space of the model's categorical predictions. If the vocabulary has size V, then every next-token distribution is a probability vector in Δ^{V-1}. A different tokenizer changes both the labels and the sequence length. The same raw text can become many character tokens or fewer subword tokens.

This matters because the model does not optimize probability of raw Unicode strings directly. It optimizes probability of token sequences. If two tokenizers represent the same string with different token sequences, they create different conditional prediction problems.

1.4.4 Character versus subword tokenization

A character tokenizer is transparent:

```
"cat" -> [id("c"), id("a"), id("t")]
```

The advantage is conceptual simplicity. The disadvantage is longer sequences. A subword tokenizer learns frequent chunks:

```
"attention" -> [id("att"), id("ention")]  # illustrative, not exact
```

The advantage is shorter sequences and reusable pieces for rare words. The disadvantage is that the learned vocabulary and merge rules become part of the data pipeline.

1.4.5 Code path

The repo exposes both teaching forms:

- `CharTokenizer.from_text(text)` builds a simple character vocabulary.
- `char_tok.encode(...)` maps text to integer IDs.
- `char_tok.decode(...)` maps IDs back to text.
- `train_bpe_tokenizer([text], vocab_size=80)` trains a tiny BPE tokenizer for demonstration.

The next code cell checks a round trip for the character tokenizer and compares character-token length with BPE-token length on the same text prefix.

> **Mathematician's lens.** Tokenization chooses a finite alphabet and represents strings as sequences over that alphabet. The language model is then a probability model over this discrete sequence space.
>
> **ML practitioner's warning.** Tokenization is not harmless preprocessing. It changes sequence length, vocabulary size, rare-word behavior, special-token handling, and the cost of attention through `T`.

1.4.6 Self-checks

1. If a tokenizer doubles the average number of tokens per document, what happens to the size of a full attention score matrix for the same raw document?
2. Why is token ID order not meaningful as a numeric order?

3. Which model tensors change shape when `vocab_size` changes?

```
from llm_from_scratch.tokenization import CharTokenizer, train_bpe_tokenizer

text = (PROJECT_ROOT / "data" / "tiny_corpus.txt").read_text(encoding="utf-8")
char_tok = CharTokenizer.from_text(text)
ids = char_tok.encode(text[:80])
round_trip = char_tok.decode(ids)

bpe_tok = train_bpe_tokenizer([text], vocab_size=80)
bpe_ids = bpe_tok.encode(text[:80]).ids

assert round_trip == text[:80]
char_tok.vocab_size, len(ids), len(bpe_ids), ids[:12], bpe_ids[:12]
```

1.4.7 What Tokenization Does And Does Not Decide

Tokenization decides the model's discrete alphabet. It does not decide the architecture. A transformer can use character tokens, BPE tokens, byte tokens, or other discrete units. The rest of the model only sees integer IDs.

However, tokenization changes three mathematical quantities that matter throughout the notebook:

1. **Vocabulary size V.** This sets the dimension of the output simplex and the width of the final logit vector.
2. **Sequence length T.** This sets the number of positions in a context window and the size of the attention matrix.
3. **Empirical target distribution.** Token frequencies and conditional patterns depend on the tokenizer.

The connection to attention is especially direct. Vanilla causal self-attention builds a score matrix with shape `(T, T)` per head per example. If tokenization makes sequences longer, the quadratic term grows quickly:

$$\text{scores per head} = T^2.$$

So a tokenizer can affect both statistical efficiency and hardware cost.

1.4.8 Connection to the toy LLM

`ModelConfig(vocab_size=...)` fixes the size of:

- `TinyGPT.token_embedding`, shape `(V, C)`,
- `TinyGPT.lm_head`, shape `(C, V)` conceptually, though PyTorch stores linear weights as `(V, C)`,
- cross-entropy logits, final axis length `V`.

The context length appears as `block_size`. It fixes:

- the maximum input window length,
- the position embedding table shape `(T_max, C)`,
- the registered causal mask shape `(1, 1, T_max, T_max)` in `CausalSelfAttention`.

Mathematician's lens. Tokenization is a representation choice that changes the coordinate system of the discrete sample space before any neural network map is applied.

ML practitioner's warning. Comparing models with different tokenizers can be misleading if you compare only per-token loss. A token is not a stable unit across tokenizers.

1.4.9 CS124 Tokenization Lens: Types, Instances, Unknowns, And BPE

A useful CS124 distinction is:

Term	Meaning in this notebook
Type	A vocabulary entry, such as a character, byte, word, or subword unit.
Instance	One occurrence in running text.
Token ID	The integer emitted by a tokenizer for one type occurrence.
Vocabulary size V	The number of possible token IDs the model can predict.

This matters because word vocabularies do not close neatly. As the corpus grows, more word types appear. A fixed word-level vocabulary therefore creates an unknown-word problem: a test string can contain a word type absent from training.

Subword tokenization is the compromise. A BPE-style tokenizer has two conceptually separate parts:

1. **Learner:** start from small units, count adjacent pairs in a corpus, and iteratively merge frequent pairs into longer units.
2. **Encoder:** apply the learned merge rules to segment new text into known token IDs.

Byte-level BPE pushes the base alphabet all the way down to bytes. Because there are only 256 possible byte values, every Unicode string can be represented without an unknown token. Learned merges then recover common multi-byte characters, morphemes, words, or word fragments when the corpus supports them.

The underappreciated consequence is that tokenization changes both statistics and compute. It changes the empirical distribution over labels, the sequence length T, and the attention cost proportional to T^2. A tokenizer is therefore not a preprocessing detail. It is part of the modeling contract.

Source note: this integrates CS124/Speech and Language Processing material on types, instances, vocabulary growth, unknown words, BPE learner/encoder structure, and byte-level BPE from `../data/cs124/slp3_chapters/2.pdf` and `../data/cs124/lectures/tokens_jan26.pdf`.

1.4.10 Classical String Algorithms Before Token IDs

A transformer sees integer IDs, but the input begins as strings. CS124 puts several older NLP tools before neural modeling because they still define the contract that produces those IDs.

Tool	Question it answers	Why it still matters
Unicode and UTF-8	what characters or bytes does this string contain?	different encodings change the raw units available to tokenization
Regular expressions	what surface patterns should be matched or segmented?	dataset cleaning, token boundaries, and filters often start here

Tool	Question it answers	Why it still matters
Edit distance	how many insertions, deletions, or substitutions separate two strings?	spelling variation and approximate matching are not solved by token IDs alone
Noisy-channel correction	which intended word likely produced this observed typo?	probability can combine a source model with an error model
BPE	which adjacent units should be merged into reusable subword types?	subword vocabularies are learned from corpus statistics before LM training

This matters for LLMs because tokenization is not semantic understanding. BPE can make frequent strings cheap to represent, and byte-level BPE can represent arbitrary Unicode text, but the model still receives a sequence of IDs whose boundaries were chosen by an algorithm outside the transformer.

A practical rule: when a model behaves oddly around whitespace, casing, spelling, code, emoji, or non-English text, inspect the string-to-token layer before blaming attention.

Source note: this section integrates the CS124 words/tokens/edit-distance material from `../data/cs124/slp3_chapters/2.pdf`, `../data/cs124/slp3_chapters/D.pdf`, `../data/cs124/lectures/med25.pdf`, and `../data/cs124/pa1-regular-expressions/`.

1.5 3. Embeddings And Next-Token Language Modeling

1.5.1 Mathematical object

Token IDs are labels from a finite set. The integer `17` does not mean "larger" than token `8` in a semantic sense. The model therefore learns an embedding map

$$E : \mathcal{V} \to \mathbb{R}^C.$$

If the vocabulary has size V, this map can be represented by a matrix

$$W_E \in \mathbb{R}^{V \times C}.$$

The embedding of token i is the i-th row:

$$E(i) = W_E[i, :].$$

Equivalently, let $e_i \in \mathbb{R}^V$ be a one-hot row vector with a 1 in position i. Then

$$e_i W_E = W_E[i, :].$$

The lookup implementation is faster than explicitly forming one-hot vectors, but the mathematical object is a learned linear map from one-hot coordinates into a continuous vector space.

1.5.2 Dimensions and tensor shapes

For token IDs `ids` with shape `(B, T)`, PyTorch `nn.Embedding(V, C)` returns

```
x = embedding(ids)
x.shape == (B, T, C)
```

The toy `TinyGPT` also learns a position embedding table with shape `(block_size, C)`. The initial residual stream is

$$x_{b,t,:} = W_E[x_{b,t}], : + W_P[t, :].$$

So token identity and position identity are both represented in the same C-dimensional residual stream.

1.5.3 Statistical and geometric role

Embeddings let the model learn representation geometry. Two tokens whose rows are close in the embedding space may behave similarly under later linear maps. This similarity is not imposed by the token IDs; it is learned from the training objective.

The embedding table is trained by the same next-token loss as the rest of the model. If changing an embedding row helps predict future tokens in contexts where that token appears, gradient descent adjusts that row.

1.5.4 Code path

In the code cell below, `torch.nn.Embedding(V, C)` illustrates the lookup. In the model, the corresponding parameters are:

```
self.token_embedding = nn.Embedding(config.vocab_size, config.n_embd)
self.position_embedding = nn.Embedding(config.block_size, config.n_embd)
```

and the forward pass begins with:

```
positions = torch.arange(seq_len, device=idx.device)
x = self.token_embedding(idx) + self.position_embedding(positions)
```

The position embedding tensor of shape `(T, C)` broadcasts across the batch dimension when added to token embeddings of shape `(B, T, C)`.

> **Mathematician's lens.** An embedding table is a function from a finite set into a vector space. It turns discrete symbols into coordinates where linear algebra can operate.
>
> **ML practitioner's warning.** Embedding geometry is learned from data and objective, not from pure meaning. Nearby vectors can reflect syntax, frequency, formatting, artifacts, or spurious correlations.

1.5.5 Self-checks

1. If `V=50_000` and `C=768`, how many trainable scalar parameters are in the token embedding table?
2. Why is an embedding lookup equivalent to one-hot matrix multiplication?
3. In `TinyGPT`, why can the position embedding of shape `(T, C)` be added to a token embedding tensor of shape `(B, T, C)`?

1.5.6 Static Embeddings Versus Contextual Representations

The embedding table begins as a type-level lookup table. Token ID i always selects the same row $W_E[i, :]$ before the transformer has seen context. This is analogous to a static embedding dictionary: one type, one vector.

After transformer layers, the vectors are no longer static type embeddings. The residual stream at position t has been changed by position information, attention to earlier tokens, MLP transformations, layer normalization, and residual updates. It is better to think of the final hidden vector as a contextual token-instance representation:

$$\text{same token ID} + \text{different context} \Rightarrow \text{different hidden vector.}$$

This distinction prevents a common confusion. The row in the embedding matrix is a learned starting representation for a token type. The hidden state produced after attention is a representation of a token instance in a particular context.

Similarity in these spaces should also be interpreted carefully. Cosine similarity is a normalized dot product; it measures geometric angle, not truth, entailment, or probability. It is useful because it reduces raw vector-length effects, but it is still only one measurement on a learned representation space.

Source note: this follows the CS124 vector-semantics and contextual-embedding materials, especially the embedding-matrix shape view in SLP3 Chapter 6 and the static-vs-contextual distinction in SLP3 Chapter 10 and `../data/cs124/lectures/ContextualEmbeddings25.pdf`.

```
B, T, V, C = 2, 4, 10, 6
ids = torch.tensor([[1, 2, 3, 4], [4, 3, 2, 1]])
embedding = torch.nn.Embedding(V, C)
x = embedding(ids)

assert ids.shape == (B, T)
assert x.shape == (B, T, C)
x.shape
```

1.5.7 Logits And Label Shifting

1.5.8 Mathematical object

A raw token sequence

$$(x_1, x_2, \dots, x_{T+1})$$

contains T next-token prediction examples:

$$(x_1) \mapsto x_2,$$

$$(x_1, x_2) \mapsto x_3,$$

and so on through

$$(x_1, \dots, x_T) \mapsto x_{T+1}.$$

In a fixed-width training window, this is implemented by shifting labels one position to the left relative to the input:

```
text IDs:  [10, 11, 12, 13, 14]
input:     [10, 11, 12, 13]
target:    [11, 12, 13, 14]
```

At input position t, the target is the next token.

1.5.9 Dimensions and tensor shapes

The data helper `get_batch(tokens, block_size, batch_size, device)` returns:

```
x.shape == (B, T)
y.shape == (B, T)
```

with

```
y[:, :-1] == x[:, 1:]
```

for each sampled window. The final target in each row is the token immediately after the input window in the original 1D token stream.

The model then returns

```
logits.shape == (B, T, V)
```

and cross-entropy compares `logits[b, t, :]` with `y[b, t]`.

1.5.10 Empirical risk from label shifting

The shifted-label construction turns one contiguous sequence into many supervised examples while preserving causal structure. The batch loss is

$$L(\theta) = \frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} -\log p_\theta(y_{b,t} \mid x_{b,\le t}).$$

The notation $x_{b,\le t}$ matters. The transformer receives the full input window, but causal masking prevents position t from reading positions greater than t. Without that mask, the position trying to predict `y[b,t]` could indirectly see future tokens in `x[b,t+1:]`, making the objective invalid.

1.5.11 Code path

The label shift is created in `src/llm_from_scratch/data.py`:

```
x = torch.stack([tokens[i : i + block_size] for i in starts])
y = torch.stack([tokens[i + 1 : i + block_size + 1] for i in starts])
```

The loss is consumed in `TinyGPT.forward` by flattening logits and targets.

> **Mathematician's lens.** Label shifting is the step that converts density estimation on sequences into a sum of conditional classification losses.
>
> **ML practitioner's warning.** Off-by-one label errors can produce a model that trains but learns the wrong task. Always inspect a small `x, y` pair by hand.

1.5.12 Self-checks

1. For a token stream of length 100 and `block_size=8`, how many possible starting positions are valid in this repo's `get_batch` logic?
2. Why does next-token prediction require a causal mask once the whole input window is processed in parallel?
3. Which tensor axis indexes vocabulary logits, and which tensor indexes target class IDs?

```
from llm_from_scratch.data import get_batch

all_ids = torch.arange(20)
x_batch, y_batch = get_batch(all_ids, block_size=5, batch_size=3, device=device)

assert x_batch.shape == y_batch.shape == (3, 5)
assert torch.equal(y_batch[:, :-1], x_batch[:, 1:])
x_batch, y_batch
```

1.6 4. Attention From Raw Tensors

1.6.1 Mathematical object

Attention is a learned weighted average. Each token position builds three vectors:

- a query q_t: what this position is looking for,
- a key k_s: what source position s offers as an address,
- a value v_s: what source position s contributes if selected.

For a single head with per-head dimension D, collect these vectors into matrices

$$Q, K, V \in \mathbb{R}^{T \times D}.$$

The unnormalized score from receiving position t to source position s is a dot product:

$$a_{t,s} = q_t \cdot k_s.$$

All pairwise scores are contained in

$$A = QK^T \in \mathbb{R}^{T \times T}.$$

After scaling, masking, and row-wise softmax, each row becomes a probability distribution over source positions:

$$W_{t,:} = \text{softmax}\left(\frac{A_{t,:}}{\sqrt{D}} + \text{mask}_{t,:}\right).$$

The output at position t is

$$o_t = \sum_{s=1}^{T} W_{t,s} v_s.$$

Thus attention is a row-stochastic matrix W multiplying values:

$$O = WV.$$

"Row-stochastic" means $W_{t,s} \geq 0$ and $\sum_s W_{t,s} = 1$ for every receiving position t.

1.6.2 Dimensions and tensor shapes

The teaching function `scaled_dot_product_attention` uses tensors shaped `(B, T, D)`:

```
scores = q @ k.transpose(-2, -1)  # (B, T, T)
weights = stable_softmax(scores, dim=-1)
out = weights @ v                 # (B, T, D)
```

The module `CausalSelfAttention` handles multiple heads. It starts with `x.shape == (B, T, C)`, applies one linear layer to obtain concatenated Q, K, and V, then reshapes each to `(B, H, T, D)` where `D = C // H`. The score tensor has shape `(B, H, T, T)`.

1.6.3 Statistical and modeling role

Attention lets the conditional distribution at one position depend on earlier token representations. Without attention or recurrence, each position would be transformed mostly independently after embeddings. With causal self-attention, position t can form a context-dependent representation by reading positions $s \leq t$.

This is still part of next-token prediction. Attention does not introduce a new loss. It changes the function class used to compute logits.

1.6.4 Optional entropy-regularized lens

For a fixed query q_t, keys k_s, and values v_s, the attention weights can be viewed as a soft selection over the simplex. The softmax distribution solves an optimization problem of the form

$$w^* = \arg\max_{w \in \Delta^{T-1}} \left\{ \sum_s w_s a_s + \tau H(w) \right\},$$

where a_s are scores, $H(w) = -\sum_s w_s \log w_s$ is entropy, and τ controls softness. This lens says attention prefers high-score positions while entropy prevents an immediate hard argmax. You do not need this view to implement attention, but it explains why softmax attention is a differentiable relaxation of selection.

1.6.5 Code path

The notebook first uses `scaled_dot_product_attention` from `src/llm_from_scratch/functional.py`. The full model uses `CausalSelfAttention` in `src/llm_from_scratch/attention.py`.

> **Mathematician's lens.** Attention constructs a data-dependent linear operator W on the sequence positions. The values are averaged linearly, but the averaging weights are nonlinear functions of queries and keys.
>
> **ML practitioner's warning.** Attention weights are not automatically explanations. They are internal mixture weights that can be affected by scaling, masking, heads, layer composition, and downstream projections.

1.6.6 Self-checks

1. If `q` and `k` have shape `(B, T, D)`, why does `q @ k.transpose(-2, -1)` have shape `(B, T, T)`?
2. Which axis should softmax normalize in the attention score tensor?
3. Why is `weights @ v` a weighted average over source positions rather than over channels?

1.6.7 Scaling And The Causal Mask

1.6.8 Variance scaling

Assume, as a rough initialization model, that components of q and k are independent with mean 0 and variance 1. The dot product is

$$q \cdot k = \sum_{r=1}^{D} q_r k_r.$$

Each product $q_r k_r$ has mean 0. Under the simple independence assumptions, its variance is approximately 1. Therefore

$$\mathrm{Var}(q \cdot k) \approx D.$$

As D grows, raw scores become large in magnitude. Large-magnitude logits push softmax toward saturated distributions, where most probability sits on one entry and gradients can become small or unstable. Scaling by $1/\sqrt{D}$ gives

$$\mathrm{Var}\left(\frac{q \cdot k}{\sqrt{D}}\right) \approx 1.$$

This is why attention uses

$$\frac{QK^T}{\sqrt{D}}.$$

1.6.9 Causal masking

For next-token prediction, receiving position t may use positions $s \leq t$ but not positions $s > t$. The causal mask is a lower-triangular Boolean matrix:

$$M_{t,s} = \begin{cases} 1, & s \leq t, \\ 0, & s > t. \end{cases}$$

The implementation replaces forbidden scores with $-\infty$ before softmax:

$$\mathrm{softmax}([\ldots, -\infty, \ldots]) = [\ldots, 0, \ldots].$$

So future positions receive exactly zero attention probability.

1.6.10 Dimensions and code path

The helper `causal_mask(T)` returns a `(T, T)` Boolean lower-triangular mask. In `CausalSelfAttention`, the mask is stored as

```
self.mask.shape == (1, 1, block_size, block_size)
```

so it can broadcast over `(B, H, T, T)` attention scores. At runtime the module slices it to the actual sequence length:

```
scores = scores.masked_fill(~self.mask[:, :, :seq_len, :seq_len], float("-inf"))
```

The next code cell verifies three contracts:

1. output shape is `(B, T, D)`,
2. attention weights have shape `(B, T, T)`,
3. masked future weights are zero and each row sums to 1.

Mathematician's lens. The mask restricts the support of each row distribution. Attention is still a probability vector, but it lives on a smaller simplex determined by causal order.

ML practitioner's warning. Forgetting the causal mask can produce deceptively low training loss because the model can access information that should be unavailable at inference time.

1.6.11 Self-checks

1. Under the simple variance assumptions, why does the unscaled dot product variance grow with D?
2. Why does setting a score to $-\infty$ before softmax force probability zero?
3. In a multi-head score tensor `(B, H, T, T)`, which two axes does the causal mask constrain?

```
from llm_from_scratch.functional import causal_mask, scaled_dot_product_attention

B, T, D = 1, 5, 8
q = torch.randn(B, T, D)
k = torch.randn(B, T, D)
v = torch.randn(B, T, D)
mask = causal_mask(T)
out, weights = scaled_dot_product_attention(q, k, v, mask)

assert out.shape == (B, T, D)
assert weights.shape == (B, T, T)
assert torch.allclose(weights[0].triu(1), torch.zeros(T, T).triu(1))
assert torch.allclose(weights.sum(dim=-1), torch.ones(B, T))
out.shape, weights[0]
```

1.6.12 The `T x T` Bottleneck

The score matrix has one row for every receiving position and one column for every source position. That is T^2 scores per head per example. With batch and heads included, the attention score tensor has shape

$$(B, H, T, T).$$

The memory for scores alone is proportional to

$$BHT^2.$$

The matrix multiplication cost also grows quadratically in T for each head. This is one of the central bottlenecks that motivates many alternatives to vanilla attention.

1.6.13 Why the bottleneck matters

If `T=2048`, one attention head forms $2048^2 = 4,194,304$ scores for one example. If the model has many heads and layers, the total number of intermediate scores becomes large. During training, additional memory is needed for gradients and optimizer state. During decoding with a KV cache, the pattern changes: one new query attends over cached keys and values, but the cache itself grows with sequence length.

1.6.14 Connection to later architecture ideas

Many post-transformer or transformer-variant ideas can be read as answers to a precise question:

- Can we avoid comparing every position with every other position?
- Can we share keys and values across heads?
- Can we compress history into a state rather than keep all pairwise interactions?
- Can retrieval supply relevant context without extending the model's internal sequence indefinitely?
- Can a different objective learn representations with fewer labeled or next-token samples?

The important habit is to identify the bottleneck before judging the architecture name.

> **Mathematician's lens.** Full attention constructs a dense kernel matrix over token positions. Sparse, local, recurrent, and state-space variants change the structure or representation of that operator.
>
> **ML practitioner's warning.** Reducing asymptotic cost does not automatically improve wall-clock speed or model quality. Hardware kernels, memory layout, batch size, and data distribution often decide whether a theoretical improvement matters.

1.6.15 Residual-Stream View: What Attention Moves

The transformer block is easiest to read if you track the residual stream: one vector per token position, repeatedly modified by sublayers. Most sublayers act positionwise. The MLP transforms each token vector independently. Layer norm normalizes each token vector independently across its channel dimension.

Attention is the exceptional operation. It lets a receiving position build an update from source positions. In the CS124 framing, query, key, and value are three projected roles of the same residual-stream vectors:

- the **query** is what the receiving position asks for;
- the **key** is what each source position offers for matching;
- the **value** is the information copied, averaged, or mixed into the receiving position.

The causal mask determines which source positions are allowed to write into each receiver. Without the mask, a decoder-only model can leak future target information into the current representation. With the mask, attention becomes controlled information movement from the prefix into the current position.

This view also explains why attention dominates many architecture discussions. It is the main cross-position mixer, and its score tensor grows as `(B, H, T, T)`.

Source note: this integrates the CS124 transformer lecture's residual-stream, Q/K/V, causal attention, and quadratic-cost framing plus the PA6a attention scaffold in `../data/cs124/pa6a-transformers/model.py`.

```
for T_value in [128, 512, 2048, 8192]:
    print(T_value, "full attention scores per head:", T_value * T_value)
```

1.7 5. The Transformer Block

1.7.1 Mathematical object

A decoder-only transformer block is a residual map on a sequence of vectors. If the residual stream is

$$x \in \mathbb{R}^{B \times T \times C},$$

then one pre-norm block has the form

$$x' = x + \mathrm{Attn}(\mathrm{LN}_1(x)),$$

$$x'' = x' + \mathrm{MLP}(\mathrm{LN}_2(x')).$$

The output shape is still `(B, T, C)`. This shape preservation is why blocks can be stacked.

1.7.2 Layer normalization

For a vector $u \in \mathbb{R}^C$ at one batch-position pair, layer norm computes

$$\mu = \frac{1}{C} \sum_{i=1}^{C} u_i,$$

$$\sigma^2 = \frac{1}{C} \sum_{i=1}^{C} (u_i - \mu)^2,$$

and returns

$$\mathrm{LN}(u)_i = \gamma_i \frac{u_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

The learned vectors $\gamma, \beta \in \mathbb{R}^C$ let the model rescale and shift normalized activations.

1.7.3 MLP and GELU

The feed-forward network acts independently at each token position:

$$\mathrm{MLP}(u) = W_2 \ \mathrm{GELU}(W_1 u + b_1) + b_2.$$

In this repo, the hidden width is `4 * n_embd`, so the MLP expands from C to $4C$ and projects back to \$C‘. The nonlinearity is GELU, a smooth activation commonly used in transformer blocks.

1.7.4 Residual learning

A residual connection lets a sublayer learn an update rather than a full replacement:

$$x \mapsto x + f(x).$$

If a block is not useful early in training, it can learn an update near zero while preserving the existing residual stream. This helps gradient flow through deep compositions because there is a direct additive path across layers.

1.7.5 Pre-norm and post-norm structure

Transformer blocks are often described by whether normalization happens before or after a sublayer. In post-norm form, the schematic update is

$$x_{\text{out}} = \text{LN}(x + F(x)).$$

In pre-norm form, the update is

$$x_{\text{out}} = x + F(\text{LN}(x)).$$

Both preserve the residual-stream shape `(B, T, C)`, but they put the normalization in different places in the composed map. This repo uses pre-norm.

The toy block is pre-norm because normalization happens before each sublayer:

```
x = x + self.attn(self.ln_1(x))
x = x + self.ff(self.ln_2(x))
```

This is implemented in `GPTBlock.forward`. Pre-norm often makes optimization more stable for deeper transformers because the sublayers receive normalized inputs while the residual stream remains the main information path.

1.7.6 Multi-head attention inside the block

The attention module first projects the residual stream to Q, K, and V with a single linear layer. If `C=32` and `H=4`, then `D=8`. Shapes move as follows:

```
x:      (B, T, C)
qkv:    (B, T, 3C)
q,k,v:  (B, T, C)
heads:  (B, H, T, D)
scores: (B, H, T, T)
out:    (B, T, C)
```

Each head attends in a lower-dimensional subspace. The head outputs are concatenated back to width `C` and mixed by the output projection.

1.7.7 Code path

The relevant files are:

- `src/llm_from_scratch/attention.py`: `CausalSelfAttention`,
- `src/llm_from_scratch/model.py`: `FeedForward`, `GPTBlock`, `TinyGPT`,

- `src/llm_from_scratch/configs.py`: `ModelConfig` shape constraints.

The next code cell instantiates `TinyGPT`, sends a random `(B, T)` token-ID batch through it, and checks that logits have shape `(B, T, V)`.

> **Mathematician's lens.** A transformer stack is a composition of shape-preserving residual maps on $\mathbb{R}^{T\times C}$, with attention mixing across the sequence axis and MLPs mixing across the channel axis.
>
> **ML practitioner's warning.** Residual connections and layer norm help optimization, but they do not fix bad data, label leakage, an impossible context length, or a learning rate that is too large.

1.7.8 Self-checks

1. Why must `n_embd` be divisible by `n_head` in `ModelConfig`?
2. Which sublayer mixes information across token positions, and which sublayer acts positionwise?
3. Why does the residual addition require the sublayer output to have shape `(B, T, C)`?

```
from llm_from_scratch.model import TinyGPT, count_parameters

cfg = ModelConfig(vocab_size=64, block_size=16, n_embd=32, n_head=4, n_layer=2, dropout=0.0)
model = TinyGPT(cfg)
idx = torch.randint(0, cfg.vocab_size, (2, cfg.block_size))
logits, loss = model(idx, idx)

assert logits.shape == (2, cfg.block_size, cfg.vocab_size)
assert loss is not None and loss.item() > 0
count_parameters(model), logits.shape, loss.item()
```

1.7.9 Parameter Count Intuition

The toy model has the same categories of parameters as a larger GPT-style model:

Parameter group	Shape intuition	Role
Token embedding	`(V, C)`	map token IDs to vectors
Position embedding	`(block_size, C)`	encode absolute position
QKV projection	roughly `(C, 3C)` per block	create queries, keys, values
Attention output projection	roughly `(C, C)` per block	mix concatenated heads
MLP first projection	roughly `(C, 4C)` per block	expand channel dimension
MLP second projection	roughly `(4C, C)` per block	return to residual width
Layer norm parameters	vectors of length `C`	scale and shift normalized activations
LM head	conceptually `(C, V)`	map hidden states to vocabulary logits

In `TinyGPT`, the final language-model head shares weights with the token embedding table:

```
self.lm_head.weight = self.token_embedding.weight
```

This is called weight tying. It means the same learned token vectors used to read input IDs are also used, transposed conceptually, to score output vocabulary items. Weight tying reduces parameters and links input and output representation geometry.

1.7.10 Scaling intuition

Increasing `n_embd` raises the cost of most matrix multiplications roughly quadratically in width. Increasing `n_layer` repeats the block more times. Increasing `block_size` increases position embedding size and the maximum attention mask, and it can increase attention cost quadratically in the sequence length actually used. Increasing `vocab_size` grows the embedding and output dimensions.

The toy model is not useful because it is large. It is useful because the categories of computation match the larger family.

> **Mathematician's lens.** Parameter count is a crude measure of the dimension of the hypothesis class. Architecture determines how those degrees of freedom are shared across positions and composed.
>
> **ML practitioner's warning.** More parameters can improve fit, but they also increase memory, compute, overfitting risk on small data, and the need for careful evaluation.

1.8 6. Training: Loss, Batches, Optimizer, And Overfitting

1.8.1 Mathematical object

Training minimizes an empirical risk. For a language-model batch, the risk estimate is

$$L(\theta) = \frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} -\log p_\theta(y_{b,t} \mid x_{b,\le t}).$$

The full dataset objective is approximated by randomly sampled minibatches. Each batch gives a noisy estimate of the gradient of the full empirical risk.

1.8.2 Gradient descent conceptually

Plain gradient descent updates parameters by

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k).$$

Stochastic gradient descent uses a minibatch estimate of ∇L. AdamW, used in this repo, adds adaptive moment estimates and decoupled weight decay. At a high level:

- it tracks a moving average of gradients,
- it tracks a moving average of squared gradients,
- it rescales updates by the estimated gradient scale,
- it applies weight decay separately from the gradient step.

You do not need the full AdamW derivation to understand the toy training loop, but you should know that the optimizer is not changing the objective. The objective is still average next-token cross-entropy. AdamW is a rule for moving through parameter space.

1.8.3 Training loop structure

The helper `train_steps` in `src/llm_from_scratch/train.py` does the standard loop:

1. sample `x, y` with `get_batch`,
2. run `logits, loss = model(x, y)`,

3. clear old gradients,
4. call `loss.backward()`,
5. optionally clip gradients,
6. call `optimizer.step()`,
7. periodically estimate train and validation loss.

The code cell below uses `overfit_tiny_batch`, which intentionally trains on one fixed random batch. This is a diagnostic, not a real training goal.

1.8.4 Tiny-batch overfit as a diagnostic

A sufficiently expressive model should be able to drive loss down on a tiny fixed batch. If it cannot, likely causes include:

- labels are shifted incorrectly,
- gradients are not flowing,
- the model is in the wrong mode,
- the learning rate is too small or too large,
- logits and targets have incompatible shapes,
- the loss is averaged or masked incorrectly.

The diagnostic asks a narrow question: can the system reduce the objective on data it sees repeatedly? Passing that test does not prove generalization. Failing it is a serious implementation warning.

1.8.5 Optimization loss versus generalization

Optimization asks whether the training objective decreases. Generalization asks whether performance transfers to held-out data from the intended distribution. These are different questions. In language modeling, validation loss estimates the average negative log-likelihood on tokens not used for parameter updates.

> **Mathematician's lens.** Training is numerical optimization of a high-dimensional nonconvex empirical objective. The gradient is local information; the dataset and architecture define the objective landscape.
>
> **ML practitioner's warning.** A training curve can improve while the model learns leakage, memorizes duplicates, overfits formatting artifacts, or fails the behavior you actually care about.

1.8.6 Self-checks

1. Why does a minibatch gradient estimate introduce noise compared with the full empirical gradient?
2. What does tiny-batch overfitting prove, and what does it not prove?
3. Where in `train_steps` are gradients cleared, computed, and applied?

```
from llm_from_scratch.train import overfit_tiny_batch

set_seed(123)
small_cfg = ModelConfig(vocab_size=16, block_size=8, n_embd=32, n_head=4, n_layer=1, dropout=0.0)
small_model = TinyGPT(small_cfg)
x = torch.randint(0, small_cfg.vocab_size, (8, small_cfg.block_size))
y = torch.randint(0, small_cfg.vocab_size, (8, small_cfg.block_size))
first_loss, last_loss = overfit_tiny_batch(small_model, x, y, steps=30, lr=1e-2)

assert last_loss < first_loss
```

```
first_loss, last_loss
```

1.8.7 Generalization Versus Memorization

Overfitting a tiny batch is a diagnostic. It is not the final goal. A model that memorizes eight windows has shown that the loss, gradients, and optimizer can interact productively. It has not shown that the learned conditional distribution works on new text.

Validation loss is the first guard against fooling yourself. Training loss asks:

> How well did the model fit token positions used for updates?

Validation loss asks:

> How well does the model predict held-out token positions sampled from a similar distribution?

If the training loss decreases while validation loss stops improving or gets worse, the model may be overfitting. If both losses are high, the model may be underfit, poorly optimized, too small, trained on too little data, or facing a harder tokenization problem.

1.8.8 Perplexity

When the loss is average negative log-likelihood measured in natural-log units, perplexity is

$$\text{perplexity} = \exp(L).$$

A loss of 0 would give perplexity 1, meaning the model assigns probability 1 to every observed target. A loss of $\log V$ corresponds to uniform guessing over V tokens and gives perplexity V.

Perplexity is useful because it is tied directly to the probabilistic objective. It is still not a complete behavioral evaluation. For instruction-following, factuality, retrieval, or safety, task-specific evaluations are needed.

1.8.9 Code path

The repo's `evaluate.py` contains `estimate_loss` for train/validation averages and `perplexity(loss)` for the exponential conversion. Generation later can look subjectively better or worse than validation loss suggests because decoding changes how probabilities are sampled.

> **Mathematician's lens.** Generalization is about expected loss under a data-generating distribution, while training observes only a finite sample from that distribution.
>
> **ML practitioner's warning.** Validation sets can leak, become overused, or fail to represent deployment data. Treat validation as evidence, not as proof of broad competence.

1.9 7. Generation And Evaluation

1.9.1 Mathematical object

After training, the model defines conditional distributions

$$p_\theta(x_{t+1} \mid x_{\le t}).$$

Generation uses these conditionals to sample a sequence. Starting from a prompt $x_{1:m}$, repeat:

1. compute logits $z \in \mathbb{R}^V$ for the final position,
2. transform logits into a probability vector,
3. draw or choose the next token,
4. append it to the context.

The model parameters θ do not change during generation.

1.9.2 Prefill versus decoding

A production decoder-only model separates inference into two phases.

Prefill processes the prompt tokens in parallel and builds the per-layer KV cache. If the prompt has length m, the model computes keys and values for those m positions once. This phase is still causal: token i cannot attend to token $j > i$. But because the whole prompt is known, the implementation can pack the prompt into matrix operations.

Decoding generates one new token at a time. At each step, the model computes the new token's query, key, and value, appends the new key and value to the cache, attends the query to cached positions, samples or selects the next token, and repeats.

The toy `generate` function does not implement a KV cache. It recomputes the model over the cropped context each step because that is simpler for teaching. The mathematical contract is the same conditional distribution, but the systems contract is different: real serving avoids recomputing past keys and values.

1.9.3 Temperature scaling

Temperature rescales logits before softmax:

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \qquad \tau > 0.$$

If $\tau < 1$, logit differences become larger and the distribution becomes sharper. If $\tau > 1$, differences shrink and the distribution becomes flatter. As $\tau \to 0$, sampling approaches argmax selection if there is a unique largest logit. As $\tau \to \infty$, the distribution approaches uniform over unfiltered tokens.

1.9.4 Greedy and beam decoding

The simplest deterministic decoding rule is greedy search:

$$\widehat{x}_{t+1} = \arg\max_i p_\theta(i \mid x_{\le t}).$$

Greedy search is fast, but a locally best token can lead to a poor full sequence. Beam search keeps the top K partial sequences by accumulated log probability:

$$\log p(y_{1:i} \mid x) = \log p(y_{<i} \mid x) + \log p(y_i \mid x, y_{<i}).$$

Beam search is a search procedure over sequence prefixes. Top-k and top-p sampling are different: they prune or truncate the next-token distribution and then sample from the remaining categorical distribution.

1.9.5 Top-k filtering

Top-k keeps only the k largest logits and sets all others to $-\infty$ before softmax. If S_k is the set of kept token IDs, then

$$p_i = 0 \quad \text{for } i \notin S_k.$$

The remaining probabilities are renormalized by softmax.

1.9.6 Top-p filtering

Top-p, also called nucleus sampling, sorts tokens by probability and keeps the smallest prefix whose cumulative probability exceeds p. This makes the number of allowed tokens depend on the distribution shape. A peaked distribution may keep few tokens; a flat distribution may keep many.

1.9.7 Categorical sampling

After filtering and softmax, generation samples from a categorical distribution over token IDs. In PyTorch, the toy code uses `torch.multinomial(probs, num_samples=1)`.

1.9.8 Code path

`src/llm_from_scratch/generate.py` contains:

- `top_k_filter`,
- `top_p_filter`,
- `sample_next_token`,
- `generate`.

The `generate` function crops context to `model.config.block_size`, runs the model, takes `logits[:, -1, :]`, samples one next token, and concatenates it to `idx`.

> **Mathematician's lens.** Decoding is a procedure for drawing a path through a sequence of conditional categorical distributions. Temperature and filtering transform the inference-time distribution but do not alter the learned parameterized family.
>
> **ML practitioner's warning.** Decoding settings can hide or amplify model weaknesses. A better sample does not mean the model learned a better distribution; it may only mean the sampling procedure avoided low-quality regions.

1.9.9 Self-checks

1. Why does `generate` use only `logits[:, -1, :]` at each step?
2. What happens to probabilities of filtered-out tokens after setting logits to $-\infty$?
3. Why does changing temperature not require backpropagation?

1.9.10 Decoding Policies: Greedy, Sampling, Temperature, And Truncation

Training defines a conditional distribution. Generation requires a policy for turning that distribution into the next token.

Policy	Rule	Benefit	Failure mode
Greedy decoding	choose `argmax` token	deterministic and simple	can be repetitive and brittle
Multinomial sampling	sample from the full softmax distribution	respects model uncertainty	can sample low-quality tail tokens
Temperature sampling	divide logits by τ before softmax	controls sharpness/entropy	bad settings distort behavior
Top-k sampling	keep only the `k` highest-logit tokens	removes low-probability tail	fixed `k` may be too narrow or too broad
Top-p sampling	keep the smallest set whose mass exceeds `p`	adapts to distribution shape	still needs tuning and evaluation

Temperature is especially easy to connect to the softmax math. If logits are z, generation samples from

$$\text{softmax}(z/\tau).$$

For $0 < \tau < 1$, large logits become larger relative to the rest, so the distribution sharpens. For $\tau > 1$, differences shrink and the distribution flattens. The model parameters have not changed. Only the decoding policy has changed.

This is why qualitative samples are not a pure model metric. They depend on prompt, random seed, temperature, truncation, stopping rules, and the model itself.

Source note: this follows SLP3 Chapter 7 and CS124's LLM lecture on greedy decoding, random sampling, temperature, and the PA6a `generate` implementation.

```
from llm_from_scratch.generate import generate
from llm_from_scratch.evaluate import perplexity

set_seed(123)
gen_cfg = ModelConfig(vocab_size=20, block_size=8, n_embd=16, n_head=4, n_layer=1, dropout=0.0)
gen_model = TinyGPT(gen_cfg)
prompt = torch.zeros((1, 3), dtype=torch.long)
out_ids = generate(gen_model, prompt, max_new_tokens=5, temperature=1.0, top_k=5)

assert out_ids.shape == (1, 8)
perplexity_of_one_nat = perplexity(1.0)
out_ids, perplexity_of_one_nat
```

1.9.11 Why Qualitative Samples Can Mislead

Generated text is useful for intuition, but it is noisy evidence. A few good samples do not prove that a model has a good distribution. A few bad samples do not fully characterize it either. Sampling is stochastic, prompt-sensitive, and decoding-sensitive.

For the tiny curriculum model, generated text will often look poor. That is expected. The model is small, the data are tiny, and the training budget is minimal. The goal is to understand the mechanics, not to produce a useful assistant.

1.9.12 Evaluation hierarchy

A rigorous evaluation habit separates several questions:

Question	Example evidence	Failure mode
Does the code run?	notebook execution, tests	says little about learning quality
Does the objective decrease?	training loss	can reflect memorization or leakage
Does held-out likelihood improve?	validation loss, perplexity	may not match downstream behavior
Does task behavior improve?	task-specific metrics	can overfit benchmark format
Does deployment behavior hold?	monitoring, red-team tests, latency checks	distribution shift and system effects

The toy repo mostly covers the first three. Modern LLM practice adds the rest.

> **Mathematician's lens.** A sample is one realization from a distribution. Estimating properties of the distribution requires repeated measurements and a defined statistic.
>
> **ML practitioner's warning.** Cherry-picked generations are not evaluation. Always ask how prompts were selected, how many samples were tried, and what metric or rubric was fixed before looking at outputs.

1.10 8. Toy Supervised Fine-Tuning

1.10.1 Mathematical object

Supervised fine-tuning changes the data distribution and often the formatting. Instead of raw text windows alone, examples may have instruction-response structure:

```
### Instruction:
<user task>

### Response:
<desired answer>
```

The model is still usually trained with next-token prediction. What changes is which sequences are sampled and which token positions are included in the loss.

Let $m_{b,t} \in \{0, 1\}$ be a loss mask. The masked empirical risk is

$$L(\theta) = \frac{\sum_{b,t} m_{b,t} \left[-\log p_\theta(y_{b,t} \mid x_{b,\le t})\right]}{\sum_{b,t} m_{b,t}}.$$

Prompt positions can have mask 0, while response positions have mask 1. This trains the model primarily to generate the response given the prompt, rather than spending loss budget on reproducing prompt text.

1.10.2 Dimensions and tensor shapes

For a batch of formatted examples:

```
x.shape == (B, T)
y.shape == (B, T)
loss_mask.shape == (B, T)
per_token_loss.shape == (B, T)
```

The mask has the same token-position shape as targets. It weights which next-token losses contribute to the scalar objective.

1.10.3 Statistical role

SFT does not create a new architecture. It changes the empirical distribution and the target behavior. The model sees examples where the desired continuation has the style and structure of an answer. Masking makes the objective more precise: condition on the instruction, optimize the response.

1.10.4 Code path

The repo keeps SFT as a toy orientation topic. `toy_instruction_examples()` returns a few prompt-response pairs. The notebook formats them and then demonstrates masked averaging directly with tensors. A production SFT pipeline would need a tokenizer, special tokens, padding, sequence packing, mask construction, validation splits, and careful data audits.

> **Mathematician's lens.** Loss masking changes the measure used in the empirical average. The model class may be unchanged, but the weighted objective is different.
>
> **ML practitioner's warning.** Formatting is part of the training signal. Inconsistent templates, wrong masks, or response leakage can teach brittle behavior even when the loss decreases.

1.10.5 Self-checks

1. If every mask value is 1, how does masked loss relate to ordinary cross-entropy?
2. Why might prompt tokens be included in context but excluded from loss?
3. What happens if the response mask is shifted by one position incorrectly?

```
from llm_from_scratch.data import toy_instruction_examples

examples = toy_instruction_examples()
formatted = []
for prompt, response in examples[:2]:
    formatted.append(f"### Instruction:\n{prompt}\n\n### Response:\n{response}")

formatted
```

1.10.6 Loss Masking Intuition

The code cell below uses fixed per-token losses so the masking formula is visible. With

```
per_token_loss = [2.0, 2.0, 1.0, 0.5, 0.25]
loss_mask      = [0,   0,   1,   1,   1]
```

the masked average is

$$\frac{1 + 0.5 + 0.25}{3}.$$

The first two token positions still exist in the sequence. The model may attend to them if they are in context. They simply do not contribute to the scalar loss.

1.10.7 Weighted empirical risk

More generally, masks need not be only 0 or 1. A weighted loss could use $m_{b,t} \geq 0$:

$$L(\theta) = \frac{\sum_{b,t} m_{b,t} \ell_{b,t}(\theta)}{\sum_{b,t} m_{b,t}}.$$

Binary prompt-response masking is the simplest case. The denominator matters: without it, examples with more unmasked tokens would automatically contribute larger total loss.

1.10.8 Connection to implementation practice

PyTorch's built-in cross-entropy can ignore a special target index, or you can compute unreduced per-token losses and multiply by a mask. The toy notebook uses the second idea directly because it makes the mathematics explicit.

> **Mathematician's lens.** Masking is multiplication by an indicator function inside an empirical integral or sum.
>
> **ML practitioner's warning.** Always inspect masks on decoded examples. A correct-looking tensor shape does not prove the right tokens are supervised.

```
targets = torch.tensor([[5, 6, 7, 8, 9]])
loss_mask = torch.tensor([[0, 0, 1, 1, 1]], dtype=torch.float32)
per_token_loss = torch.tensor([[2.0, 2.0, 1.0, 0.5, 0.25]])
masked_average = (per_token_loss * loss_mask).sum() / loss_mask.sum()

assert abs(masked_average.item() - ((1.0 + 0.5 + 0.25) / 3)) < 1e-6
masked_average.item()
```

1.11 9. Translation To Hugging Face And PyTorch Abstractions

After building from scratch, library abstractions become easier to interpret. The point is not that libraries are unimportant. The point is that abstractions are safer when you know the contracts they hide.

From scratch	Library abstraction	Mathematical or tensor contract hidden
text list	`datasets.Dataset`	finite sample from a data distribution; split and map operations
`CharTokenizer` or BPE helper	`Tokenizer` / `AutoTokenizer`	text-to-ID map, vocabulary, special tokens, offsets
`ModelConfig`	`GPT2Config` or another config object	dimensions `V`, `T`, `C`, `H`, number of layers
`TinyGPT`	`AutoModelForCausalLM`	embedding, blocks, logits, optional loss
manual loop	`Trainer` or Accelerate loop	batching, optimizer, scheduler, device placement, checkpointing
manual sampling	`.generate()`	temperature, top-k, top-p, beams, stopping criteria

1.11.1 What should transfer from the toy implementation

When you use a library model, the same questions still matter:

1. What shape are `input_ids`, `attention_mask`, and labels?
2. Does the model shift labels internally, or must the data collator do it?
3. Which positions are ignored in the loss?
4. What does the model return: logits, loss, hidden states, attentions?
5. Which generation settings modify decoding but not parameters?
6. Is evaluation measuring likelihood, task accuracy, preference, latency, or something else?

1.11.2 Code path

The next code cell instantiates a tiny Hugging Face `GPT2LMHeadModel` and a `datasets.Dataset`. It does not train a real model. It is a translation check: the objects are larger abstractions around the same ideas already built by hand.

> **Mathematician's lens.** A library config is a compact specification of dimensions and module choices. It is not a replacement for the underlying map from token sequences to conditional distributions.
>
> **ML practitioner's warning.** High-level APIs can silently handle shifting, masking, padding, and generation defaults. Read the shape and loss contracts before trusting results.

1.11.3 Encoder, Decoder, And Encoder-Decoder Families

This notebook builds a decoder-only, GPT-style causal language model. That is one transformer family, not the whole transformer family.

Family	Attention pattern	Typical objective	Common use
Encoder-only	bidirectional self-attention over the input	masked-token prediction or representation learning	classification, retrieval encoders, tagging features
Decoder-only	causal self-attention over the prefix	next-token prediction	completion, chat base models, autoregressive generation
Encoder-decoder	encoder self-attention plus decoder causal self-attention and cross-attention	conditional generation with teacher forcing	translation, summarization, input-conditioned generation

The distinction is about which information each position is allowed to use. A decoder-only LM predicts the next token from the prefix, so future tokens must be masked. An encoder-only masked LM can look left and right because it is not trained to generate left-to-right continuations in the same way. An encoder-decoder model first represents a source sequence, then lets the decoder generate a target sequence while attending back to the encoded source.

When you see `AutoModelForCausalLM`, assume the decoder-only contract unless the model documentation says otherwise. When you see BERT-style encoders, do not assume they can be used as drop-in autoregressive generators. When you see translation models, look for the encoder outputs, decoder inputs, label shifting, and cross-attention boundary.

Source note: this map comes from the transformer, masked-LM, and machine-translation sources in `../data/cs124/slp3_chapters/8.pdf`, `../data/cs124/slp3_chapters/10.pdf`, `../data/cs124/slp3_`

chapters/12.pdf, ../data/cs124/slp3_slides/transformer_jan26.pdf, and ../data/cs124/slp3_slides/mlmjan25.pdf.

```
from datasets import Dataset
from tokenizers import Tokenizer
from tokenizers.models import BPE
from transformers import GPT2Config, GPT2LMHeadModel

hf_dataset = Dataset.from_dict({"text": ["attention reads earlier tokens", "loss trains logits"]})
hf_config = GPT2Config(vocab_size=128, n_positions=32, n_embd=64, n_layer=2, n_head=4)
hf_model = GPT2LMHeadModel(hf_config)

len(hf_dataset), Tokenizer(BPE()).get_vocab_size(), hf_model.num_parameters()
```

1.12 10. Quantization: Lower Precision Representations

1.12.1 Mathematical object

Quantization maps a real-valued tensor to a finite set of integer levels. A common affine quantizer uses integers in an interval

$$q \in \{q_{\min}, q_{\min} + 1, \dots, q_{\max}\}.$$

For a real value x, quantization computes

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s} + z\right), q_{\min}, q_{\max}\right),$$

and dequantization reconstructs

$$\hat{x} = s(q - z).$$

Here $s > 0$ is the scale and z is the zero-point. The scale is the bin width in real units. The zero-point chooses which integer represents real zero.

1.12.2 Choosing scale and zero-point

For asymmetric unsigned quantization with `num_bits=8`, often

$$q_{\min} = 0, \qquad q_{\max} = 255.$$

Given observed tensor range $[x_{\min}, x_{\max}]$, the toy helper uses

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}},$$

and

$$z = \text{round}\left(q_{\min} - \frac{x_{\min}}{s}\right),$$

then clips z into the integer range.

Symmetric quantization instead fixes zero-point at 0 and uses the maximum absolute value. This is often convenient for signed integer arithmetic, but it may waste levels when the real range is very asymmetric.

1.12.3 Error sources

Quantization introduces two basic errors:

1. **Rounding error.** Values inside the represented range are rounded to the nearest grid point. The error is usually bounded by about half a scale unit before other effects.
2. **Clipping error.** Values outside the represented range are clipped to `qmin` or `qmax`, which can create large errors for outliers.

The tradeoff is visible in the scale. A large range increases scale and therefore rounding error for typical values. A small range improves resolution but clips outliers.

1.12.4 Per-tensor versus per-channel

Per-tensor quantization uses one scale and zero-point for the whole tensor. Per-channel quantization uses separate parameters along an axis. For a weight matrix, different output channels can have very different ranges, so per-channel quantization can reduce error. It costs extra metadata and can require different kernels.

1.12.5 KV-cache memory formula

During autoregressive decoding, transformers often cache keys and values for previous tokens. A simple KV-cache memory estimate is

$$\text{bytes} = 2 \cdot L \cdot H \cdot D \cdot T \cdot B \cdot \text{bytes per value},$$

where:

- the factor 2 is for keys and values,
- L is number of layers,
- H is number of heads,
- D is head dimension,
- T is cached sequence length,
- B is batch size.

The repo helper `estimate_kv_cache_bytes` implements exactly this product.

The textbook adds a useful axis-level interpretation. A standard multi-head cache stores keys and values for every layer, every attention head, every cached token, and every head dimension. Ignoring the factor of 2 and batch/bytes constants, the shape cost is

$$O(L \cdot H \cdot D \cdot T).$$

Multi-query attention shares one key/value set across all query heads, reducing the head factor from H to 1 in the KV cache. Grouped-query attention uses G key/value groups, giving an intermediate cache cost

$$O(L \cdot G \cdot D \cdot T), \qquad 1 \leq G \leq H.$$

This is a serving-time memory and bandwidth tradeoff, not a change to the next-token loss.

1.12.6 Code path

`src/llm_from_scratch/quantization.py` contains:

- `QuantizationParams`,
- `quantize_tensor`,
- `dequantize_tensor`,
- `fake_quantize_tensor`,
- `quantize_per_channel`,
- `quantization_error`,
- `estimate_kv_cache_bytes`.

The next code cell quantizes a tiny tensor, dequantizes it, measures error, and computes a KV-cache estimate.

> **Mathematician's lens.** Quantization replaces a continuum with a finite lattice and studies the approximation error induced by projection onto that lattice.
>
> **ML practitioner's warning.** Quantization quality depends on tensor distributions, calibration data, hardware kernels, and which tensors are quantized. A lower bit-width is not automatically faster or acceptable.

1.12.7 Self-checks

1. If `num_bits=4` for unsigned affine quantization, what are `qmin` and `qmax`?
2. Why can an outlier increase rounding error for many non-outlier values in per-tensor quantization?
3. In the KV-cache formula, why is there a factor of 2?

```
from llm_from_scratch.quantization import (
    quantize_tensor,
    dequantize_tensor,
    quantization_error,
    estimate_kv_cache_bytes,
)

x = torch.tensor([-1.0, -0.1, 0.0, 0.5, 1.0])
q, params = quantize_tensor(x, num_bits=8)
x_hat = dequantize_tensor(q, params)
err = quantization_error(x, x_hat)
kv_bytes = estimate_kv_cache_bytes(batch_size=1, n_layer=2, n_head=4, seq_len=128, head_dim=16,␣
 ↪bytes_per_value=2)

assert err["mae"] < 0.01
q, x_hat, err, kv_bytes
```

1.12.8 Per-Tensor Versus Per-Channel Quantization

Per-tensor quantization uses one scale for the whole tensor. Per-channel quantization uses separate scales along a chosen axis. The axis decision is a shape decision, just like choosing the softmax axis in attention.

Suppose a weight tensor has rows with very different ranges. A single scale must cover the largest range, so small-range rows get coarse resolution. Per-channel quantization can give each row its own scale:

$$q_{i,j} = \text{round}\left(\frac{x_{i,j}}{s_i} + z_i\right).$$

This often reduces reconstruction error, especially for weight matrices. The cost is extra scale and zero-point metadata and possibly more complex kernels.

1.12.9 Practical tradeoffs

Method	Benefit	Cost
Per-tensor	simple metadata and kernels	sensitive to outliers and mixed ranges
Per-channel	lower error when channels have different ranges	more metadata and axis-specific implementation
Groupwise	compromise between the two	group size becomes another hyperparameter

The toy helper `quantize_per_channel(x, axis, num_bits)` makes the axis explicit. It iterates over slices along the chosen axis and applies `quantize_tensor` to each slice.

> **Mathematician's lens.** Per-channel quantization uses a product of smaller quantizers instead of one global quantizer for the whole tensor.
>
> **ML practitioner's warning.** Quantization reports should state bit-width, symmetric/asymmetric choice, calibration method, which tensors were quantized, and the hardware path. Otherwise comparisons are underspecified.

1.13 11. Modern LLM Practice: What Gets Added Around The Core Model

The toy GPT path teaches the central language-modeling mechanism: tokenize text, embed IDs, apply causal transformer blocks, produce logits, optimize next-token cross-entropy, and decode from the resulting conditional distribution. Modern LLM systems add objectives, data pipelines, evaluation procedures, retrieval systems, compression methods, and serving constraints around that core.

1.13.1 Pretraining

Pretraining fits a broad next-token model on a large text distribution. The objective is still approximately average negative log-likelihood:

$$\mathbb{E}_{x \sim \mathcal{D}_{\text{pretrain}}}\left[\sum_t -\log p_\theta(x_{t+1} \mid x_{\le t})\right].$$

What changes is scale: data volume, model size, training time, distributed systems, filtering, deduplication, and evaluation.

1.13.2 Supervised fine-tuning

SFT changes the data distribution to curated instruction-response examples. Often the loss is masked so response tokens matter more than prompt tokens. The objective is still token-level likelihood, but the empirical distribution is now closer to desired interaction format.

1.13.3 Preference optimization

Preference methods use comparisons between candidate outputs. RLHF-style pipelines commonly train a reward model from comparisons and then update a policy model against that reward signal, often while penalizing drift from a reference model. The high-level regularized objective has the shape

$$\text{maximize reward}(x, y) - \beta \left[\log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x)\right],$$

with sign conventions depending on whether the objective is written as a maximization or minimization. The reference term discourages the policy from moving too far away from the starting model.

DPO-style methods remove the explicit reward-model training step. Given a preferred response y^+ and a rejected response y^- for prompt x, DPO compares how much more the target policy favors y^+ over y^- relative to a fixed reference policy:

$$\log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\text{ref}}(y^+ \mid x)} - \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\text{ref}}(y^- \mid x)}.$$

A logistic loss then increases this margin. The common idea is that desired behavior is not fully specified by next-token likelihood on raw text. Preference data introduces another statistical signal over complete outputs.

1.13.4 Prompting and in-context learning

Prompting changes the input sequence, not the parameters. In-context learning is the special case where the prompt includes demonstrations or instructions that specify a task. Mathematically, the model is still sampling from $p_\theta(\text{continuation} \mid \text{prompt})$. Operationally, the prompt acts like task conditioning. This is why prompt length, example order, formatting, and irrelevant context can change behavior without any gradient update.

1.13.5 Retrieval-augmented generation and tool use

Retrieval does not necessarily change model parameters. It changes the conditional context at inference time by fetching documents and inserting them into the prompt. Mathematically, the model samples from

$$p_\theta(x_{\text{answer}} \mid x_{\text{question}}, r_1, \ldots, r_k),$$

where the r_i are retrieved passages. The system quality depends on both retrieval and generation.

A useful way to view RAG is problem decomposition:

1. retrieve or select evidence from an external source,
2. condition the language model on that evidence,
3. generate an answer that is faithful to the provided context.

Tool use is a related decomposition, except the model or surrounding system may call an external function during inference: a calculator, database, search engine, simulator, or API. The key difference is that a tool call can produce new observations during the generation process, while basic RAG usually inserts retrieved text before answer generation.

The failure modes follow the decomposition. Retrieval can return irrelevant evidence. The prompt can over- or under-constrain the model. The model can ignore evidence, overuse weak evidence, or fail to abstain when context is insufficient.

1.13.6 Evaluation

Evaluation must match the claim. Held-out language-model loss measures likelihood. Multiple-choice benchmarks measure a narrow decision format. Human preference ratings measure judged output quality under a sampling and prompt protocol. Serving metrics measure latency, throughput, memory, and reliability.

1.13.7 Inference serving: prefill, decode, and scheduling

Modern serving distinguishes prefill from decoding. Prefill builds the KV cache for the prompt with parallel matrix operations. Decoding updates the cache one token at a time and is often constrained by memory bandwidth because every generated token reads cached keys and values. This distinction explains why a long prompt and a long generated answer stress different parts of the system.

Serving systems also batch requests. Request-level batching groups full requests together, while continuous batching updates the active batch as requests arrive or finish. Paged KV-cache systems split cache storage into blocks so variable-length sequences do not require one large contiguous allocation. These are systems techniques around the same attention tensors studied earlier.

1.13.8 Serving and constraints

Serving adds constraints that are mostly absent from the toy notebook:

- memory for weights and KV cache,
- latency per generated token,
- batching and scheduling,
- quantization and kernels,
- context-window management,
- monitoring and rollback.

These constraints connect back to the same shapes: `V`, `T`, `C`, `H`, `D`, number of layers, and bytes per value.

1.13.9 Code path

The `stage_map` in the next cell is intentionally small. It is not a full implementation of modern LLM practice. It is a map from each practice area back to the objective, data distribution, or system constraint it modifies.

> **Mathematician's lens.** Modern LLM work can often be classified by which object changes: the model family, the training distribution, the loss, the inference context, the decoding rule, or the deployment constraint.
>
> **ML practitioner's warning.** Avoid vague claims like "the model understands" without stating the objective, data, evaluation, and failure cases. Modern systems are composites, and failures can come from any component.

1.13.10 Self-checks

1. Which parts of SFT change the model architecture, and which parts change the data/objective?
2. Why is retrieval an inference-time conditioning strategy rather than necessarily a parameter update?

3. What metric would you use for memory pressure during long-context decoding?

1.13.11 Post-Training Stage Boundary

CS124 separates the base language model from post-training. The boundary is useful because each stage changes a different object.

Stage	What changes	Training or inference signal	What it cannot guarantee alone
Pretraining	base model weights	next-token likelihood over broad text	instruction following, safety, freshness, task reliability
Supervised fine-tuning	model weights	curated instruction-response examples	preference quality beyond the demonstrations
Preference alignment	model weights or policy behavior	comparisons, rewards, or direct preference losses	truthfulness without evidence or perfect tool use
Test-time compute	inference procedure	extra reasoning steps, search, verification, or tool calls	better base knowledge or fixed bad retrieval
Retrieval/tool wrapping	external context and actions	search results, tool observations, memory state	correctness if sources or tools are wrong

The undergraduate mistake is to call all of this "the LLM." The cleaner view is a pipeline: a pretrained conditional distribution is adapted by later data and then embedded in an inference system. If behavior changes because you added search, memory, a system prompt, or a verifier, the base next-token objective did not suddenly become a database or a planner. The system around it changed.

Source note: this section follows the CS124 post-training chapter `../data/cs124/slp3_chapters/9.pdf`, the 2025/2026 final LLM lectures, and the PA7 agent materials.

1.13.12 Retrieval, Tools, And Memory Are Outside The Base LM

A base decoder-only language model maps context tokens to next-token probabilities. Modern applications often wrap that model in additional systems:

System piece	Mathematical or engineering role	Not the same as
Retrieval	map a query to external documents or chunks	changing model weights
Tool use	choose and call external functions	next-token prediction alone
Memory	persist state across interactions	the current context window
Web search	access time-sensitive external information	parametric knowledge
Agent loop	coordinate reasoning traces, tool calls, observations, and final responses	a single forward pass

The distinction matters. If a model answers from retrieved context, the answer depends on the retriever and source text. If an agent calls a function, correctness depends on tool schemas, arguments, side effects, permissions, and observations. If memory is used, the system now has a state-management problem in addition to a language-modeling problem.

The CS124 PA7 agent assignment makes this boundary concrete: the agent decides which tool is relevant, calls it, observes the result, and then produces a response. That trajectory is a system behavior built around an LM, not a property of the LM alone.

Source note: this integrates PA7's tool-use, web-search, memory, and nondeterminism framing, plus Lab 3's distinction between lexical tf-idf retrieval and dense retrieval.

1.13.13 Sparse Retrieval, Dense Retrieval, And RAG Failure Points

Retrieval is its own model or algorithmic subsystem. A useful first split is sparse versus dense retrieval.

Retriever type	Representation	Strength	Common failure
Sparse lexical retrieval	term-count vectors with tf-idf, BM25, or related weights	exact words, rare terms, transparent inverted indexes	misses paraphrases and semantic matches with little word overlap
Dense retrieval	learned embedding vectors for queries and documents	can match related meanings without exact surface overlap	opaque similarities, domain drift, embedding bias, hard-to-debug misses

RAG composes retrieval with generation:

$$\text{question} \rightarrow \text{retriever} \rightarrow \text{passages} \rightarrow \text{LM context} \rightarrow \text{answer}.$$

That decomposition gives you a debugging checklist. A bad RAG answer might come from missing source documents, poor chunking, weak query rewriting, a sparse/dense mismatch, bad ranking, context-window packing, prompt confusion, or generation that ignores the retrieved evidence. The final answer is produced by the LM, but the evidence available to the LM is controlled upstream.

For notebook 99, the key lesson is modest: retrieval changes the context, not the pretrained weights. A source-grounded system should therefore report both the model output and the retrieval evidence used to condition it.

Source note: this section comes from `../data/cs124/slp3_chapters/11.pdf`, `../data/cs124/slp3_slides/ir_nov25.pdf`, `../data/cs124/lectures/ir_jan17_2026.pdf`, `../data/cs124/labs/Lab3_InformationRetrieval.md`, and `../data/cs124/pa3-information-retrieval/`.

```
stage_map = {
    "pretraining": "learn next-token prediction over broad text",
    "supervised fine-tuning": "teach instruction-response formats",
    "preference optimization": "shape outputs using preference signals",
    "retrieval": "condition generation on fetched context",
    "quantization": "reduce memory or bandwidth with lower precision",
    "evaluation": "test behavior without fooling yourself",
}
stage_map
```

1.14 12. Beyond Transformers And World Models

The attention section exposed the dense `(T, T)` score matrix. That matrix is the reference point for many architecture questions. Beyond-transformer work is easier to read when you ask which bottleneck is being targeted.

1.14.1 Sparse and local attention

Sparse attention changes the allowed attention graph. Instead of every position reading every earlier position, each position may read a local window or a selected pattern of positions. If a local window has size w, score count becomes closer to

$$O(Tw)$$

instead of

$$O(T^2).$$

This targets compute and memory. The risk is that important long-range dependencies may be inaccessible or require many layers to propagate.

1.14.2 Grouped-query and multi-query attention

During decoding, keys and values are cached. Standard multi-head attention stores a separate key/value stream for every head. If there are H heads, D dimensions per head, L layers, and T cached positions, the head-related cache cost is proportional to

$$L \cdot H \cdot D \cdot T.$$

Multi-query attention keeps separate query heads but shares one key/value stream across all heads. Its cache cost is proportional to

$$L \cdot 1 \cdot D \cdot T.$$

Grouped-query attention divides the query heads into G groups. Each group has one shared key/value stream, so the cost is proportional to

$$L \cdot G \cdot D \cdot T.$$

When $G = H$, grouped-query attention becomes ordinary multi-head attention. When $G = 1$, it becomes multi-query attention. This is a concrete example of a shape tradeoff: reduce the cache along the head axis, hopefully without losing too much expressiveness.

1.14.3 State-space and recurrent alternatives

State-space models and recurrent models try to summarize history in a state rather than materialize all pairwise token interactions. The mathematical response is different: instead of a dense attention kernel over positions, maintain and update a compressed state. This targets long-context efficiency, but the representation must preserve the information needed for future predictions.

1.14.4 Retrieval, external memory, and compressed context

Retrieval systems move some memory outside the model's fixed context. Instead of storing everything in parameters or the KV cache, the system searches an external corpus and conditions the model on selected results. This targets knowledge freshness, context length, and data access, but introduces retrieval errors and citation/evidence challenges.

Long-sequence methods can also be framed as memory design. The standard KV cache is an exact memory of prior keys and values, but it grows with sequence length. Sliding-window attention keeps only recent keys and values. Compressive or recurrent memory tries to store a fixed-size summary. k-NN or retrieval memory stores external vectors and retrieves relevant ones. Each option chooses a different answer to the same question: what information from the past should be available to the next query?

1.14.5 JEPA, world-model, and representation-learning ideas

JEPA-style and world-model ideas shift attention from predicting the next token directly to learning predictive representations. The question becomes: what latent variables or representations should be preserved so that future observations, actions, or abstractions are predictable? This targets sample efficiency, planning, and representation quality, but evaluation is harder because the objective may be farther from observed text likelihood.

1.14.6 A bottleneck table

Direction	Mathematical response	Bottleneck targeted	Main risk
Sparse/local attention	restrict attention graph	compute and memory	missed long-range dependencies
Grouped-query attention	share K/V across query heads	KV-cache memory and bandwidth	reduced head expressivity
State-space/recurrent models	compress history into state	long-context scaling	state may forget needed details
Retrieval	condition on external selected text	knowledge and context limits	retrieval quality controls answer quality
JEPA/world models	predict representations or latent structure	sample efficiency and abstraction	objective/evaluation mismatch

1.14.7 Code path

The next code cell compares full attention score counts with a local window count. It is not a benchmark. It is a shape-based calculation that makes the bottleneck visible.

> **Mathematician's lens.** Architecture proposals are hypotheses about which structure in the computation can be constrained, shared, approximated, or moved outside the model without losing too much predictive power.
>
> **ML practitioner's warning.** Do not judge architecture work by name alone. Ask for the exact objective, complexity, hardware path, training data, evaluation protocol, and failure cases.

1.14.8 Self-checks

1. If local attention uses window size 256, what is the score-count ratio between full and local attention at `T=8192`?

2. Which bottleneck does KV-cache sharing target during decoding?
3. Why might a representation-learning objective be harder to evaluate than next-token likelihood?

```
sequence_lengths = [128, 512, 2048, 8192]
window = 256
comparison = []
for n in sequence_lengths:
    full = n * n
    local = n * min(n, window)
    comparison.append({
        "T": n,
        "full_attention_scores": full,
        "local_window_scores": local,
        "full_to_local_ratio": full / local,
    })
comparison
```

1.14.9 How To Read A Future Architecture Paper

When you read a paper about sparse attention, state-space models, recurrent memory, retrieval, world models, or post-transformer architectures, classify the claim before judging it.

Ask:

1. **Objective:** What loss or training signal is optimized?
2. **Data distribution:** What data are used, and what filtering or supervision is assumed?
3. **Architecture:** What map is changed relative to a standard transformer?
4. **Shapes:** How do `B`, `T`, `C`, `H`, `D`, and number of layers affect memory and compute?
5. **Bottleneck:** Is the target compute, memory, sample efficiency, representation quality, planning, grounding, or evaluation?
6. **Complexity:** What is the asymptotic claim, and what constants or kernels matter?
7. **Evaluation:** Which benchmark or measurement supports the claim?
8. **Ablation:** What comparison isolates the proposed idea from scale, data, or tuning?
9. **Serving:** Does the method improve prefill, decoding, batching, KV-cache size, or hardware utilization?
10. **Failure cases:** What does the method make worse or leave unresolved?
11. **Inference phase:** Does the method primarily affect prefill, decoding, or both?
12. **Cache policy:** Does it store exact KV states, share them, compress them, page them, retrieve them, or recompute them?

This habit prevents architecture names from becoming buzzwords. It turns them into specific hypotheses about computation and learning.

1.14.10 Small measurable experiment

Before expanding scope, design the smallest experiment that could falsify the claim you care about. For this repo, examples include:

- replacing full attention with a local mask and comparing tiny validation loss at fixed parameter count,
- measuring KV-cache bytes under different head-sharing assumptions,
- checking whether a masked SFT objective supervises exactly the intended response tokens,
- comparing top-k and top-p decoding on the same fixed logits,
- measuring the cache-size difference between MHA, MQA, and GQA for the same `L`, `H`, `D`, and `T`,

- timing a toy generation loop with and without recomputing the full context, while clearly labeling that the current repo does not implement a real KV cache.

The experiment should have a clear metric, a controlled baseline, and a reason it answers the bottleneck question.

> **Mathematician's lens.** A good architecture paper proposes a constrained function class or optimization procedure and then argues that the constraint improves some tradeoff.
>
> **ML practitioner's warning.** A benchmark win without a clear comparison can be caused by more data, more compute, better tuning, leakage, or evaluation choices rather than the named architectural idea.

1.15 13. How To Study From Here

Use this notebook as the overview pass, then return to the detailed notebooks and exercises. The main skill is not memorizing formulas. It is moving cleanly between four representations of the same idea:

1. a sentence, such as "the model attends causally over previous tokens";
2. a mathematical object, such as a masked $QK^T/\sqrt{D}$ score matrix;
3. a tensor shape, such as `(B, H, T, T)`;
4. a code path, such as `CausalSelfAttention.forward`.

Recommended order:

1. Run this full notebook once without edits.
2. Re-run chapters 1-4 and write down every tensor shape by hand.
3. Complete `exercises/01_first_principles.md`.
4. Re-run chapters 5-8 and intentionally break one assertion in each chapter, then fix it.
5. Complete `exercises/02_training_and_generation.md`.
6. Spend a separate session on `10_quantization_deep_dive.ipynb` and `exercises/quantization.md`.
7. Finish with `12_beyond_transformers_and_world_models.ipynb` and `docs/notes/beyond_transformers_reading_map.md`.
8. Use Xiao and Zhu's *Foundations of Large Language Models* as a second-pass reference for topics this toy repo only orients around: pretraining variants, prompting, preference alignment, inference scheduling, and long-context memory.

1.15.1 A compact checklist for each new concept

For every new LLM concept you study, write down:

- the mathematical object,
- the tensor shape,
- the axis of any softmax, mean, mask, or gather,
- the loss or inference rule it affects,
- the exact implementation location,
- one failure mode.

For example, for attention:

- mathematical object: row-stochastic matrix over source positions,
- tensor shape: `(B, H, T, T)` for weights,
- softmax axis: source-position axis `dim=-1`,
- affects: hidden states used to compute next-token logits,

- implementation: `src/llm_from_scratch/attention.py`,
- failure mode: missing causal mask leaks future tokens.

For cross-entropy:

- mathematical object: negative log-likelihood,
- tensor shape: logits `(B, T, V)`, targets `(B, T)`,
- gather axis: vocabulary axis,
- affects: scalar training objective,
- implementation: `cross_entropy_from_logits` and `TinyGPT.forward`,
- failure mode: shifted labels or masks are wrong.

1.15.2 Final perspective

The toy model is small, but it contains the core contracts:

$$\text{tokens} \to \text{embeddings} \to \text{causal transformer states} \to \text{logits} \to \text{loss or samples.}$$

Modern systems add scale, data engineering, alignment objectives, retrieval, quantization, serving, and evaluation. Those additions are easier to understand when the core map is already clear.

> **Mathematician's lens.** Treat each model component as a map with a domain, codomain, invariants, and composition rules.
>
> **ML practitioner's warning.** Understanding the formulas is necessary but not sufficient. Real model quality depends on data, implementation details, compute, evaluation design, and deployment constraints.

1.15.3 NLP Around The Core LM

The rest of CS124 is a reminder that next-token prediction is a central objective, not the entire field. Many NLP tasks ask for explicit structure, grounding, or interaction.

Area	What it asks the system to do	Why it matters for LLM understanding
Sequence labeling	assign tags such as POS or named-entity labels to token positions	many outputs are structured labels, not free-form continuations
Parsing	recover constituency or dependency structure	grammar can involve relations that are not adjacent in the string
Information extraction	identify entities, relations, events, and times	useful answers often need schema-grounded facts
Semantic role labeling	identify who did what to whom, where, when, and how	fluent text can still confuse event roles
Sentiment and connotation	model affective meaning and social associations	word meaning includes stance, affect, and bias, not just reference
Coreference and entity linking	track mentions and connect them to entities	long outputs need stable referents across sentences

Area	What it asks the system to do	Why it matters for LLM understanding
Discourse coherence	model how sentences fit together	paragraph quality is more than local next-token plausibility
Speech	map between waveforms and linguistic units	language systems often start or end outside text
Dialogue and agents	coordinate turns, goals, tools, memory, and state	applications are loops around models, not one forward pass

This table should keep future reading grounded. If a paper claims an LLM "understands language," ask which of these behaviors was actually measured and which were only implied by fluent generation.

Source note: this boundary map comes from the later CS124/SLP3 chapters, especially `../data/cs124/slp3_chapters/17.pdf` through `25.pdf`, appendices `E.pdf` through `K.pdf`, the speech materials, and the PA7/lab agent sources.

1.15.4 CS124 Source-Backed Second Pass

The local CS124 bundle can be used as a depth pass after this notebook. The full corpus review for this pass is recorded in `docs/notes/cs124_full_corpus_review_for_notebook_99.md`.

If this notebook section feels weak	Read or inspect next	What to extract
Tokenization	`../data/cs124/slp3_chapters/2.pdf`, `../data/cs124/lectures/tokens_jan26.pdf`, `../data/cs124/pa1-regular-expressions/pa1.ipynb`	type/instance distinctions, regex tokenization, BPE learner/encoder, Unicode and byte-level tokenization
Language modeling	`../data/cs124/slp3_chapters/3.pdf`, `../data/cs124/lectures/lm_jan25.pdf`	n-gram baselines, smoothing, interpolation, train/dev/test protocol, perplexity
Embeddings	`../data/cs124/slp3_chapters/6.pdf`, `../data/cs124/lectures/vector26jan.pdf`, `../data/cs124/pa4-embeddings/pa4.ipynb`	embedding matrices, cosine, static embeddings, dense retrieval intuition
Neural networks	`../data/cs124/lectures/7_NN.pdf`, `../data/cs124/pa5-neural-networks/pa5.ipynb`	units, hidden layers, backprop, cross-entropy as training signal
Transformers	`../data/cs124/lectures/transformer_cs124_26.pdf`, `../data/cs124/pa6a-transformers/model.py`	Q/K/V roles, residual stream, causal masking, attention implementation contracts

If this notebook section feels weak	Read or inspect next	What to extract
Evaluation and generation	`../data/cs124/slp3_chapters/7.pdf`, `../data/cs124/pa6a-transformers/perplexity.py`	greedy versus sampling, temperature, perplexity computation, detector caveats
Retrieval and agents	`../data/cs124/labs/Lab3_InformationRetrieval.md`, `../data/cs124/pa7-agent/README.md`	tf-idf versus dense retrieval, tool calls, web search, memory, nondeterminism
Architecture families	`../data/cs124/slp3_chapters/8.pdf`, `../data/cs124/slp3_chapters/10.pdf`, `../data/cs124/slp3_chapters/12.pdf`	decoder-only, encoder-only, masked LM, encoder-decoder, cross-attention, teacher forcing
Post-training	`../data/cs124/slp3_chapters/9.pdf`, `../data/cs124/lectures/final124_26.pdf`, `../data/cs124/lectures/finallecture_cs124_2025.pdf`	instruction tuning, preference alignment, test-time compute, stage boundaries
Broader NLP context	`../data/cs124/slp3_chapters/17.pdf` through `25.pdf`, `../data/cs124/slp3_chapters/K.pdf`	sequence labeling, parsing, semantics, coreference, discourse, conversation, dialogue state

The study rule is the same as the notebook rule: turn every source idea into a mathematical object, a tensor shape, a code path, and one checkable question.